

Biquads: Digitale Filter auf dem Pynq-Z2 als Lehrdemonstration

Martin Schwarz

2025-07-27

Inhaltsverzeichnis

Das PYNQ-Z2 und die PYNQ-Plattform	3
Hardware-Spezifikationen des PYNQ-Z2	3
ADAU1761 Audio-Codec	4
Filterentwurf in Matlab	5
Spezifikation der Filter	6
Implementierung	8
Modellierung in Simulink	8
Direct Form II Transponiert mit Pipelining	9
Festkommaarithmetik	9
AXI4-Stream-Schnittstelle	10
Modell und Aufbau	10
HDL-Coder und Workflow Advisor	12
Modellvorbereitung für den HDL-Coder	12
Einstellungen vom HDL Workflow Advisor	13
Design in Vivado	19
Audio Codec Controller	20
Processing System (PS)	20
AXI Direct Memory Access	22
Verbindungen im Blockdesign	23
Vollendetes Design	24
Funktionsweise des generierten Filter-Codes	26
Struktur des generierten Codes	26
Umsetzung des Filters	27
Filter DUT	27
Filter-Wrapper	27
Die Filterlogik	27
Filter-Sektion	28
Echtzeitfilterung	29
Funktionstest & Filtereinbindung	29
Ursache und Bewertung	30
Analoge Audioquellen	30
Aufbau und Funktion der Lehrdemonstration	31
Pynq Overlays	32

Funktionsweise des Notebooks	33
Initialisierung des DMA-Kanals	33
DMA Übertragung	33
Öffnen und Einlesen von Wav-Dateien	35
Schreiben und Speichern von Wav-Dateien	36
Paketaufteilung	36
vollständige Filter-Anwendung	37
Kaskadierung der Filter	38
Fazit und Ausblick	39

Das PYNQ-Z2 und die PYNQ-Plattform

Die Implementierung des Filters soll auf dem PYNQ-Z2 erfolgen. Das PYNQ-Z2 ist ein FPGA-Entwicklungsboard, das speziell für die Verwendung mit der PYNQ-Plattform konzipiert wurde. Die Plattform wurde von Xilinx (heute AMD) entwickelt und verfolgt das Ziel, die Entwicklung von FPGA-Anwendungen mithilfe von Python zu vereinfachen. Anstelle über herkömmlicher Hardwarebeschreibungssprachen ermöglicht PYNQ die Nutzung von Python-Code zur Steuerung und Nutzung programmierbarer Logik auf Zynq-SoCs, direkt über Jupyter Notebooks im Webbrowser. PYNQ steht für *Python Productivity* for Zynq und ist ein Open-Source-Projekt von AMD, das die Entwicklung von Anwendungen für Zynq-SoCs erheblich vereinfacht. Ziel der Plattform ist es, die komplexe Hardwareentwicklung mit FPGAs zugänglicher zu machen. Das Board wird typischerweise mit einer speziell angepassten Linux-Distribution für PYNQ von einer microSD-Karte gestartet. Über das Netzwerk lässt sich die integrierte Jupyter-Notebook-Oberfläche aufrufen, um direkt mit Python zu arbeiten und Hardwarefunktionen anzusteuern. Die Besonderheit von PYNQ liegt darin, dass komplexe Hardwaredesigns in der programmierbaren Logik per Overlay integriert und anschließend per Python angesteuert werden können. Dies ermöglicht beispielsweise Anwendungen in der Signalverarbeitung, beim maschinellen Lernen, in der Bildverarbeitung oder Robotik, ohne tiefgehende FPGA-Kenntnisse.

Hardware-Spezifikationen des PYNQ-Z2

Die vollständigen Hardware-Spezifikationen des Boards sind auf der [offiziellen Produktseite bei AMD](#) einsehbar.

Für die hier vorgestellte Implementierung sind insbesondere folgende Spezifikationen relevant:

PYNQ-Z2	Spezifikationen:
FPGA / SoC:	Xilinx Zynq-7000 SoC (XC7Z020-1CLG400C), 256 KB On-Chip-Memory, 630 KB Block RAM, 220 DSP-Slices
Audio-I/O:	I ² S-Interface mit 24-Bit-DAC (3,5 mm TRRS-Buchse), Line-In (3,5 mm Klinke)
Netzwerk:	10/100/1000 Mbit/s Ethernet
Speicher:	512 MB DDR3-RAM (16-Bit-Bus, 1050 Mbps), 128 Mbit Quad-SPI-Flash, microSD-Karten-Slot
Taktquellen:	125 MHz für die programmierbare Logik (PL), 50 MHz für den Prozessor (PS)

ADAU1761 Audio-Codec

Auf dem PYNQ-Z2 ist ein [ADAU1761 Audio-Codec](#) von Analog Devices verbaut. In Kombination mit dem entsprechenden [Audio-Modul](#) aus der PYNQ-Bibliothek ermöglicht dieser Codec die direkte Aufnahme und Wiedergabe von Audiosignalen über die analogen Ein- und Ausgänge des Boards. Obwohl der ADAU1761 über integrierte DSP-Funktionen verfügt, werden diese nicht genutzt, da die Filterung vollständig in der programmierbaren Logik (PL) des FPGAs erfolgt. Ein besonderes Merkmal des ADAU1761 ist sein integrierter fraktionaler Phasenregelkreis (PLL). Dieser erlaubt es, aus einer Vielzahl externer Takteingänge (8 MHz bis 27 MHz) intern stabile Systemtakte zu erzeugen. Die vollständigen Spezifikationen sind im Datenblatt zu dem [ADAU1761 Audio-Codec](#) einsehbar.

ADAU1761	Spezifikationen:
Audioauflösung:	24 Bit (Sigma-Delta ADC/DAC)
Abtastraten:	8 kHz bis 96 kHz (standardmäßig 48 kHz)
Schnittstelle:	I ² S (Inter-IC Sound) – digitale Audioverbindung in programmierbarer Logik
Steuerung:	Über I ² C-Bus (in PYNQ über <i>pynq.lib.audio.AudioADAU1761</i>)
PLL-Taktgenerator:	Intern aus 8 MHz–27 MHz Eingangstakt generierbar

Filterentwurf in Matlab

In MATLAB wird der Entwurf digitaler IIR-Filter mit einer Vielzahl von Werkzeugen unterstützt, die sowohl den Entwurf als auch die Analyse und Implementierung erleichtern. Grundlage ist dabei das Prinzip, einen analogen Prototypfilter mittels Bilineartransformation in den digitalen Bereich zu überführen. Dieses Verfahren wird für gängige IIR Filtertypen wie Butterworth-, Chebyshev- und elliptische Filter eingesetzt. Die Funktionen *butter*, *cheby1*, *cheby2* und *ellip* wenden genau dieses Prinzip an. Die Vorgehensweise entspricht dabei der Filterentwicklung in Python mit der Scipy-Bibliothek.

Die direkte Umsetzung von IIR-Filtern mit Polynomkoeffizienten (b, a) kann bereits bei vergleichsweise niedrigen Ordnungen zu numerischen Instabilitäten führen. Ursache dafür sind Rundungsfehler, die durch die direkte Multiplikation aller Pole und Nullstellen im Übertragungsfunktionspolynom entstehen und dazu führen können, dass Pole außerhalb des Einheitskreises liegen. Dies beeinträchtigt die Stabilität und kann die Filterfunktion erheblich verfälschen. Um solche Instabilitäten zu vermeiden, werden IIR-Filter in der Praxis nicht als ein einziges großes Polynom, sondern als Produkt mehrerer einfacher biquadratischer Übertragungsfunktionen (Biquads) umgesetzt. Biquads sind IIR-Filterblöcke zweiter Ordnung, mit denen sich Filter höherer Ordnung durch Zerlegung und Kaskadierung in mehreren Sektionen stabil realisieren lassen. Diese Aufteilung begrenzt Rundungsfehler und verbessert die Gesamtstabilität des Filters. Dies ist insbesondere bei der Umsetzung auf Hardwareplattformen wie FPGAs oder Mikrocontrollern wichtig. MATLAB unterstützt dieses Vorgehen durch die Arbeit mit *Second-Order Sections (SOS)*. Die SOS-Darstellung wird als Matrix gespeichert, bei der jede Zeile die Zähler- und Nennerkoeffizienten einer Biquad-Sektion enthält. Zusätzlich wird ein separater Verstärkungsfaktor (*Gain*) ausgegeben, um den Frequenzgang korrekt zu skalieren. Die Entwurfsfunktionen können die Filterkoeffizienten direkt in SOS-Form bereitstellen. Alternativ lassen sich vorhandene Übertragungsfunktionen mit *tf2sos* in eine Biquad-Kaskade umwandeln.

Der Entwurfsprozess in MATLAB folgt dabei einem klaren Ablauf. Zunächst werden die Anforderungen festgelegt, wie Filtertyp, Filterordnung, Grenzfrequenzen und gegebenenfalls erlaubten Ripple im Durchlass- oder Sperrbereich. Die Grenzfrequenzen werden normiert auf die halbe Abtastrate (Nyquist-Frequenz) angegeben und müssen im Bereich von 0 bis 1 liegen. Bei Bandpass- und Bandsperrfiltern wird eine Frequenztransformation des Tiefpass-Prototyps durchgeführt, wodurch sich die Filterordnung mathematisch verdoppelt. Diese Verdoppelung muss bei der Wahl der Prototyp-Ordnung in Skripten berücksichtigt werden, da die Entwurfsfunktionen dies nicht automatisch anpassen.

Neben der rein skriptbasierten Arbeitsweise können Filter alternativ auch mit dem [Filter Designer](#) erstellt werden, der eine grafische Oberfläche für die Spezifikation der Filterparameter sowie

die Visualisierung von Frequenzgängen und Pol-Nullstellen-Diagrammen bietet. Nach dem Entwurf lassen sich die berechneten Filterkoeffizienten sowohl als Polynomkoeffizienten (b, a) als auch direkt in SOS-Form ausgeben. Die Kaskadierung in Biquads bezieht sich dabei stets auf die tatsächlich resultierende Gesamtordnung, einschließlich der Verdoppelung bei Bandpass- oder Bandsperrefiltern.

Für die Analyse stehen Funktionen wie *freqz* zur Darstellung von Amplituden- und Phasengang oder das umfassende Filter Visualization Tool (*fvtool*) zur Verfügung.

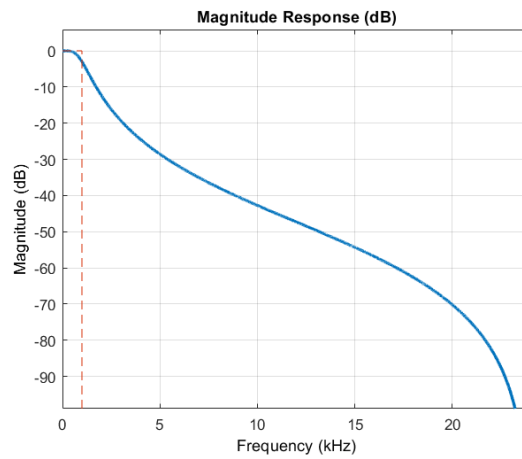
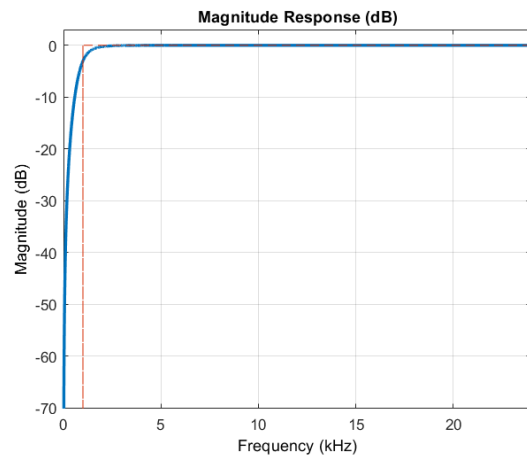
Spezifikation der Filter

Für die Demonstration wurde entschieden, alle vier grundlegenden Filtertypen als Butterworth-Filter zu realisieren. Diese besitzen den Vorteil kein Ripple im Durchlass- und Sperrbereich aufzuweisen. Gerade bei der Verarbeitung von Audiosignalen können Ripple im Frequenzgang zu unerwünschten Artefakten führen. Ein möglichst glatter Amplitudenverlauf ist daher insbesondere in der Audioanwendung vom Vorteil. Im direkten Vergleich zu elliptischen oder Chebyshev-Filtern weisen Butterworth-Filter bei gleicher Flankensteilheit eine höhere erforderliche Filterordnung auf. Diese höhere Ordnung würde in einer Hardwareumsetzung prinzipiell mehr Biquad-Stufen benötigen und damit mehr Ressourcen. Bei der Implementierung wurden die Filter auf 2. Ordnung beschränkt. Da bei einer so niedrigen Filterordnung alle Filter, unabhängig davon, ob sie als Butterworth- oder elliptischer Filter realisiert werden, ohnehin nur aus einer Biquad-Stufe bestehen, hätte die Wahl des Filtertyps keinen nennenswerten Unterschied bei den Hardwareanforderungen gemacht. Der Hauptgrund für die Entscheidung zugunsten der Butterworth-Filter liegt somit vor allem im glatten Frequenzverlauf.

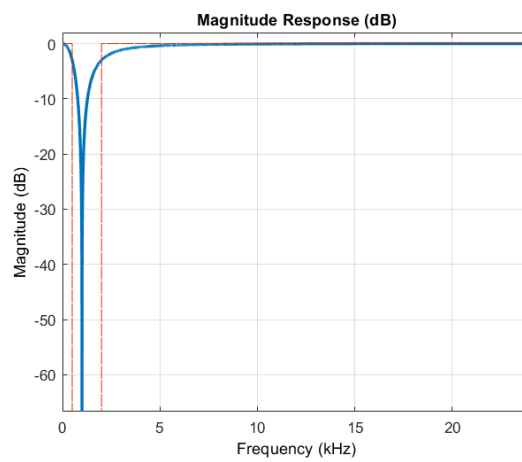
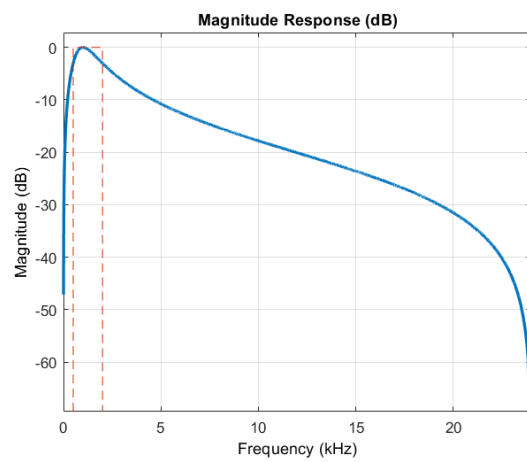
Bei der genauen Spezifikation der Filter wurde sich an dem Beispiel von [Design of Digital Audio IIR Filters](#) von Frank Schultz gehalten. Dabei ergaben sich Folgende Spezifikationen:

Filter:	Typ:	Fc:	Ordnung:
Hochpass	Butterworth	1kHz	2
Tiefpass	Butterworth	1kHz	2
Bandpass	Butterworth	500Hz - 2kHz	2
Bansstop	Butterworth	500Hz - 2kHz	2

Alle Filter wurden mit dem [Filter Designer](#) auf einer Samplefrequenz von 48kHz entworfen und die Koeffizienten als SOS-Matrix inklusive Gain in das Workspace exportiert.



(a) Digitaler Butterworth-Hochpassfilter 2. Ordnung (b) Digitaler Butterworth-Tiefpassfilter 2. Ordnung



(a) Digitaler Butterworth-Bandpassfilter 2. Ordnung (b) Digitaler Butterworth-Bandstopfilter 2. Ordnung

Implementierung

Für die Implementierung von IIR-Filtern mittels VHDL gibt es mehrere Ansätze. Eine direkte Umsetzung der Differenzengleichung eines IIR-Filters in HDL-Code ist grundsätzlich möglich. Der Nachteil dieses Verfahrens liegt jedoch im Aufwand für Test, Validierung und Debugging. Gerade im Rahmen einer Abschlussarbeit mit begrenztem Zeitrahmen ist eine manuelle Umsetzung in dieser Tiefe schwer umsetzbar, zumindest nicht in einem vergleichbaren Umfang wie bei der Verwendung des HDL Coders von MATLAB/Simulink. Der modellbasierte Ansatz über MATLAB ermöglicht eine deutlich schnellere Entwicklung und erlaubt die automatische Generierung von synthesesfähigem VHDL-Code auf Basis eines zuvor getesteten Simulink-Modells. Dies erleichtert insbesondere die Erstellung mehrerer Filtervarianten.

Für die Umsetzung eines digitalen IIR-Filters wurde daher ein Simulink-Modell entwickelt, das mithilfe des HDL Coders in VHDL-Code übersetzt wird. Ziel ist es, daraus einen Vivado-kompatiblen IP-Core zu erzeugen, der über eine AXI4-Stream-Schnittstelle in ein FPGA-Design eingebunden werden kann. Dieser modellbasierte Workflow reduziert die Entwicklungszeit und vereinfacht auch die Integration in die bestehende FPGA-Architekturen erheblich. In der Vivado Design Suite von AMD/Xilinx spielen [IP-Cores \(Intellectual Property Cores\)](#) eine zentrale Rolle. Dabei handelt es sich um wiederverwendbare Hardwaremodule, die sich modular in digitale Systeme einfügen lassen. Für die Implementierung von digitalen Filtern bieten IP-Cores einen Vorteil. Die gesamte Filterlogik kann als in sich geschlossenes Modul gekapselt werden. Dadurch wird die Funktionalität des Filters klar abgegrenzt, was sowohl die Wartung als auch die Wiederverwendbarkeit verbessert. Ein einmal entwickelte Filter-IP-Core kann unkompliziert in andere Designs oder Projekte übernommen und betrieben werden, ohne den zugrunde liegenden HDL-Code erneut anpassen zu müssen.

Modellierung in Simulink

Für die Umsetzung in Simulink wurde der in der DSP HDL Toolbox enthaltene [Biquadratic IIR \(SOS\) filter](#) verwendet. Der Vorteil dieses Blocks gegenüber einer direkten Umsetzung der transponierten Direct Form II liegt in seiner Optimierung für die HDL-Codegenerierung. Er enthält bereits eine integrierte Steuerungslogik für das valid-Signal im AXI-Stream-Protokoll und verfügt zudem über eine vorimplementierte Pipelining-Struktur. Die Probleme bei der Umsetzung einer Direct-Form-Struktur lagen hauptsächlich darin, dass bei der Bitstream-Generierung in Vivado die Timing-Anforderungen nicht eingehalten werden konnten und die Taktfrequenz des Filters entsprechend reduziert werden musste. Wird die Taktfrequenz zu niedrig gewählt, verlängert sich die Zeit der Filterung entsprechend. Hinzu kommt, dass ein einzelner Filter in dieser Struktur vergleichsweise viele Ressourcen auf dem FPGA beansprucht. Eine direkte Implementierung der

Direct Form II ohne korrektes Pipelining und weiterführende Optimierungen gestaltet sich zudem als äußerst anspruchsvoll. Aus diesem Grund fiel die Entscheidung zugunsten einer vorgefertigten Lösung aus der DSP HDL Toolbox. Trotzdem musste die Taktfrequenz letztlich auf 50 MHz begrenzt werden, da bei 100 MHz ebenfalls keine vollständige Einhaltung der Timings erreicht werden konnte.

Direct Form II Transponiert mit Pipelining

Der Filter-Block wurde auf Direct Form II Transponiert eingestellt, da diese Struktur, genau wie die klassische Direct Form II, nur eine Verzögerungskette benötigt und somit weniger Register erforderlich sind. Die transponierte Form verschiebt die Verzögerungselemente auf die Rückkopplungspfade, wodurch die kritische Pfadlänge verkürzt wird. Gleichzeitig bietet sie Vorteile in Bezug auf die numerische Stabilität und ist robuster gegenüber Quantisierungseffekten. Ergänzend ist dieser Filterblock schon gepipelined. Beim *Pipelining* werden innerhalb der Struktur gezielt Register in die kombinatorischen Signalpfade eingefügt. Dadurch werden lange Logikpfade unterbrochen, was die kritische Pfadlänge reduziert. Werte können häufiger zwischengespeichert werden, sodass pro Taktzyklus weniger Rechenoperationen durchgeführt werden müssen. Dies ermöglicht eine höhere maximale Taktfrequenz und verbessert insgesamt die Timing-Eigenschaften der Schaltung. Rückkopplungswege, die bei IIR-Filtern eine Herausforderung darstellen, können so auch bei hoher Verarbeitungsgeschwindigkeit stabil betrieben werden.

Festkommaarithmetik

Die gesamte Filterimplementierung verwendet Festkommaarithmetik, da FPGAs standardmäßig keine native Gleitkomma-Hardware besitzen. In digitalen Systemen ist die Verwendung von Festkomma eine zentrale Voraussetzung, da alle Signale und Rechenergebnisse nur mit einer endlichen Wortbreite dargestellt und verarbeitet werden können. Dies führt zwangsläufig zu Rundungs- und Quantisierungsfehlern, die insbesondere bei rekursiven Systemen wie IIR-Filtern zu Stabilitätsproblemen führen können, wenn sie nicht sorgfältig berücksichtigt werden. Ein Aspekt der Festkommaarithmetik ist die Wahl einer geeigneten Zahlendarstellung. Um die begrenzte Bitbreite optimal zu nutzen, ist eine sorgfältige Skalierung notwendig. Sie verhindert Überläufe und stellt sicher, dass die numerische Auflösung bestmöglich ausgeschöpft wird. Außerdem wirkt sich die gewählte Wortlänge auf den kritischen Pfad im digitalen System aus. Je länger die arithmetischen Operationen, desto größer sind die resultierenden Verzögerungszeiten, was wiederum die maximal erreichbare Taktfrequenz limitiert.

Angesichts dessen, wird für alle Signalpfade ein vorzeichenbehafteter 32-Bit-Festkommatyp mit 16 Nachkommabits verwendet. Dieses Format wurde sowohl in Simulink als auch in der späteren Hardwareimplementierung experimentell validiert und hat sich dabei als optimale Wahl erwiesen.

AXI4-Stream-Schnittstelle

Die Integration der Filter in das FPGA-System erfolgt über eine *AXI4-Stream-Schnittstelle*, einem Teil der standardisierten AMBA AXI4-Interface-Protokollfamilie. Diese Schnittstellenarchitektur stammt ursprünglich von Arm und wird von AMD/Xilinx weitläufig in Vivado-Designs eingesetzt. Die AXI4-Familie setzt sich dabei zusammen aus *AXI4* für adressierte Hochgeschwindigkeits-Datenübertragungen mit Burst-Unterstützung, *AXI4-Lite* für einfache Steuerregister-Kommunikation ohne Burst-Funktionalität und *AXI4-Stream* für kontinuierliche, nicht-adressierte Datenströme. Letzteres erlaubt mehrere Datenströme mit unterschiedlichen Datenbreiten über denselben Interconnect zu übertragen. Die Kommunikation basiert auf einem einfachen Handshake-Mechanismus mit den Signalen TVALID und TREADY. Ein Transfer erfolgt nur, wenn beide Signale gleichzeitig aktiv sind. Der Sender (Master) setzt TVALID sobald gültige Daten vorliegen, und der Empfänger (Slave) signalisiert mit TREADY, dass dieser bereit ist Daten zu empfangen. Der Sender darf *nicht auf TREADY warten*, bevor er TVALID aktiviert. Der Sender *muss TVALID so lange halten*, bis der Handshake abgeschlossen ist. Der Empfänger darf TREADY auch vor TVALID setzen, muss er aber nicht. Neben den verpflichtenden Signalen TVALID, TREADY und TDATA existieren im *AXI4-Stream-Protokoll* auch optionale Signale wie TID und TLAST. Das Signal TID (Transaction ID) ermöglicht es, mehrere logische Datenströme innerhalb eines gemeinsamen physischen AXI4-Stream-Kanals voneinander zu unterscheiden. Dies ist besonders relevant, wenn verschiedene Datenquellen über denselben Stream multiplexed werden. TLAST kennzeichnet bei paketbasierter Übertragung das Ende eines Datenpakets. Es wird gesetzt, sobald das letzte Datenwort eines Transfers übertragen wird. Die Verwendung von AXI4-Stream in Vivado hat den großen Vorteil, dass IP-Blöcke auf standardisierte Weise verbunden werden können. Über *AXI-Interconnects* oder *AXI SmartConnect*-Komponenten kann der Datenfluss zwischen IPs automatisch geroutet werden, was die Einbindung vereinfacht.

Modell und Aufbau

Die Numerator- und Denominator-Koeffizienten des Filters werden direkt aus dem MATLAB-Workspace übernommen und stammen aus der vorberechneten SOS-Matrix. Der Gain-Faktor wird in die Numerator-Koeffizienten eingerechnet, um eine stabile Skalierung zu gewährleisten. Dadurch wird verhindert, dass während der Berechnung Zwischenwerte entstehen, die außerhalb des darstellbaren Wertebereichs liegen.

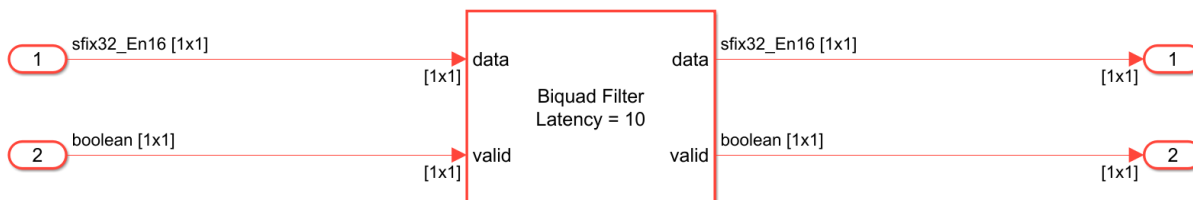
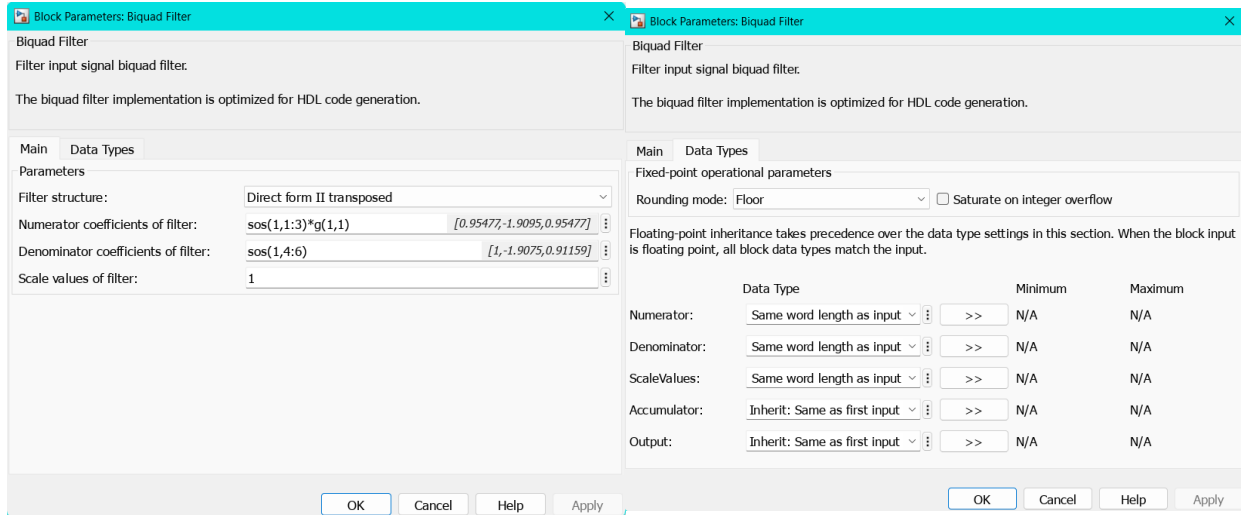


Abbildung 0.1: Simulink Filter: Subsystem

Der Filter ist in ein separates Subsystem eingebettet, um die eigentliche Filterlogik klar von zusätzlichen Test- und Steuerungselementen zu trennen. Um den Datentyp durchgängig korrekt zu setzen, wird ein Data Type Conversion-Block vor dem Filter-Subsystem eingefügt. Der Filter selbst ist so eingestellt, dass die Koeffizienten den Datentyp des Eingangssignals übernehmen. Die genauen Einstellungen sehen wie folgt aus:



(a) Biquad Filter: Main

(b) Biquad Filter: Datatypes

Ergänzend enthält das Modell Komponenten zur Test- und Signalanalyse. Dazu gehören Elemente, die Testsignale aus dem Workspace einlesen, ein gültiges AXI-Stream Valid-Signal zur Simulation erzeugen, sowie Blöcke zum Zurückschreiben der Ergebnisse ins MATLAB-Workspace.

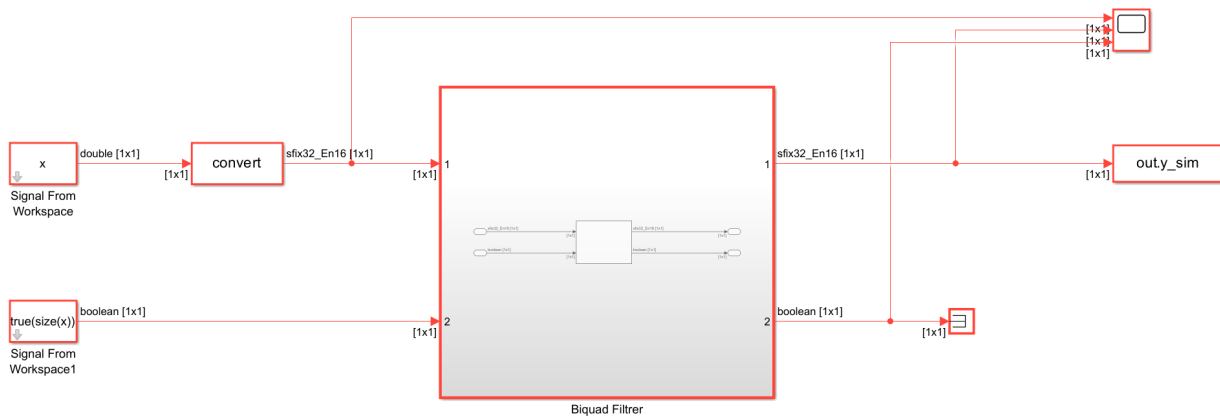


Abbildung 0.3: Simulink Filter

HDL-Coder und Workflow Advisor

Für die Überführung des Filtermodells in eine hardwaretaugliche Form kommt der *HDL Coder* von MathWorks zum Einsatz. Dieses Tool ermöglicht die automatische Generierung von synthesefähigem HDL-Code aus einem Simulink-Modell oder MATLAB-Algorithmus. Dadurch entfällt die manuelle Erstellung von VHDL- oder Verilog-Code. Der HDL Coder unterstützt einen modellbasierten Entwicklungsablauf, bei dem speziell für die Hardwaregenerierung ausgelegte, HDL-kompatible Simulink-Blöcke verwendet werden. Diese gewährleisten eine korrekte Abbildung der Signalverarbeitung im späteren FPGA-Design. Die automatisierte Codegenerierung erfolgt über den HDL Workflow Advisor, der den Nutzer schrittweise durch den gesamten Prozess führt. Abhängig von der gewählten Zielplattform wird dabei festgelegt, welche zusätzlichen Schnittstellen generiert werden sollen. So kann ein IP-Core mit AXI4-Stream- oder AXI4-Lite-Schnittstellen erzeugt werden. Die Ein- und Ausgänge des Simulink-Modells werden entsprechend den HDL-I/O-Ports oder AXI-Schnittstellen zugewiesen. Nach Abschluss der Konfiguration generiert der HDL Coder automatisch den vollständigen HDL-Code, einschließlich aller erforderlichen Constraints-Dateien (XDC) sowie optionaler Testbenches. Der gesamte Ablauf wird strukturiert durchlaufen und dokumentiert. Im Rahmen dieser Implementierung wird auf Basis der Codegenerierung ein Vivado-kompatibler IP-Core erzeugt, der in ein lokales IP-Repository abgelegt wird. Dieser IP-Core kann anschließend ohne zusätzliche Anpassungen in ein Vivado Block Design eingebunden werden. Der kombinierte Einsatz von HDL Coder und HDL Workflow Advisor ist besonders dann vorteilhaft, wenn begrenzte Erfahrung in der manuellen HDL-Entwicklung vorliegt oder eine zügige, reproduzierbare Prototypenerstellung erforderlich ist. Die automatisierte Vorgehensweise reduziert Fehlerquellen, verbessert die Wiederverwendbarkeit und gewährleistet, dass alle Schnittstellenanforderungen bereits im Modellierungsprozess berücksichtigt werden. So konnten die IIR-Filter mit AXI4-Stream-Anbindung hardwaregerecht umgesetzt werden.

Modellvorbereitung für den HDL-Coder

Damit das Simulink-Modell für den HDL-Coder verwendet werden kann, müssen einige Einstellungen in Simulink selbst durchgeführt werden. Dieses Setup lässt sich im *Command Window* in Matlab automatisch gestalten, mit der Voraussetzung, dass Vivado installiert ist. Für dieses Projekt wird Vivado 2022.1 verwendet, dies ist die aktuellste Version von Vivado welche von Pynq unterstützt wird.

Das Setup in Simulink lässt sich mit der Funktion *hdlsetup(modelname)* ausführen. Diese Funktion konfiguriert das geöffnete Modell für den HDL-Workflow, indem sie notwendige Pfade und Einstellungen für den HDL Coder ergänzt. Sie bereitet Simulink gezielt für die Codegenerierung und FPGA-Implementierung vor. Die Funktion muss pro Modell nur einmalig ausgeführt werden. Nach erfolgreicher Ausführung wird das Simulink-Modell typischerweise durch einen roten Rahmen markiert, ein Hinweis darauf, dass es für die HDL-Codegenerierung vorbereitet ist.

Die Funktion *hdlsetuptoolpath(path)* konfiguriert den Pfad zu externen Synthese-Tools, die für Codegenerierung benötigt werden. Dieser Schritt ist erforderlich, damit die HDL Workflow Advisor-Umgebung später auf die notwendigen Werkzeuge zugreifen und einen IP-Core generieren kann. Der Befehl sollte vor dem Öffnen des HDL Workflow Advisor ausgeführt werden, andernfalls kann

der Toolpfad nicht korrekt übernommen werden. Es empfiehlt sich daher, diesen Befehl in ein MATLAB-Skript zu integrieren.

Einstellungen vom HDL Workflow Advisor

Im Folgenden werden die wichtigsten Einstellungen beschrieben, die für die Erstellung und Konfiguration des IP-Cores vorgenommen wurden.

1.1 Set Target Device and Synthesis Tool

Im ersten Schritt werden im HDL Workflow Advisor die grundlegenden Parameter für die Zielhardware festgelegt. Als **Target Workflow** wird *IP Core Generation* ausgewählt, wodurch der Workflow Advisor für die Erstellung eines eigenständigen IP-Cores konfiguriert wird. Die **Target Platform** ist auf *Generic Xilinx Platform* eingestellt, es wird also keine board-spezifische Plattform gewählt, sondern eine generische Xilinx-Zielumgebung genutzt. Als **Synthesis Tool** kommt *Xilinx Vivado* in der Version 2022.1 zum Einsatz. Damit wird definiert, dass die Synthese, die Implementierung und die spätere IP-Integration in einem Vivado-Projekt erfolgen. Unter **Family**, **Device**, **Package** und **Speed** sind die Werte *Zynq*, *xc7z020*, *clg400* und *-1* gesetzt. Diese Einstellungen entsprechen den Anforderungen des PYNQ-Z2 Boards. Die Werte für **Package** und **Speed** wurden dabei automatisch erkannt und übernommen. Anschließend wird der Zielpfad angegeben, an dem die generierte IP gespeichert werden soll. Auf diese Weise ist sichergestellt, dass der IP-Core später direkt in Vivado eingebunden werden kann.

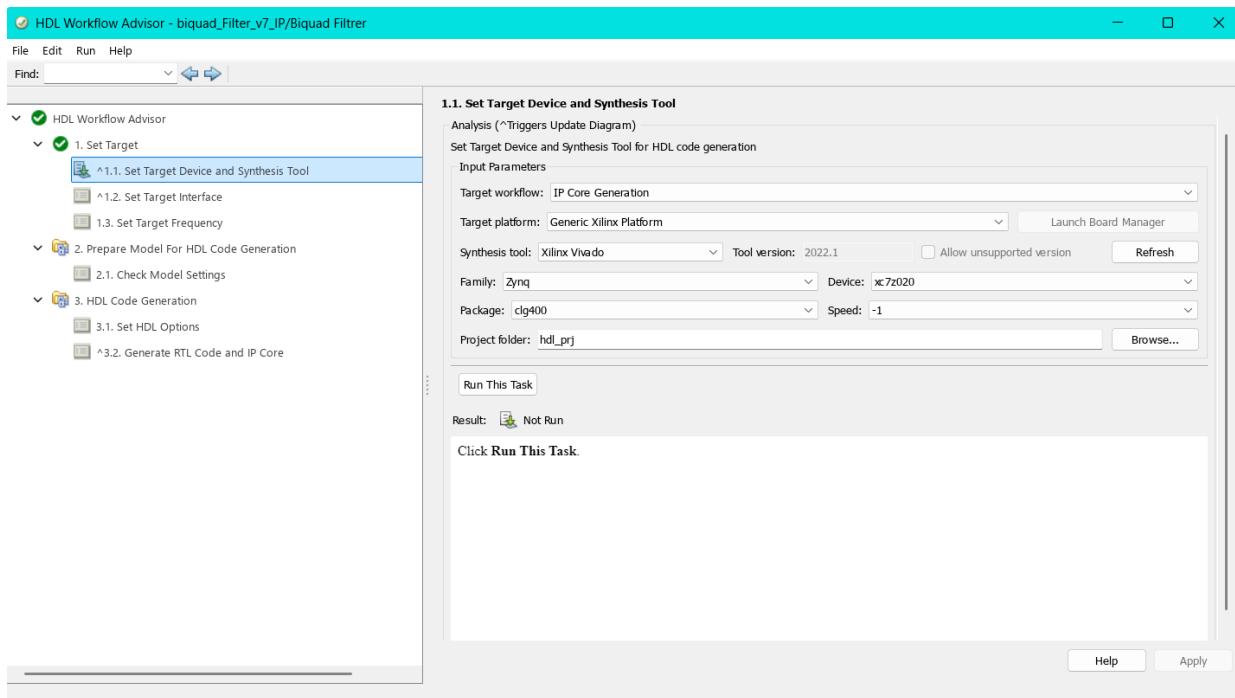


Abbildung 0.4: HDL Workflow: Target Device

1.2 Set Target Interface

In diesem Schritt werden die Ein- und Ausgänge des Simulink-Modells den Schnittstellen der AXI-Stream-Plattform zugewiesen. Die Option **Processor/FPGA Synchronization** ist auf *Free running* gesetzt, da die spätere Anwendung als kontinuierlicher Datenstrom (Streaming) ausgelegt ist. Die Übersicht in der Tabelle bietet zusätzlich einen guten Überblick über die zugewiesenen Datentypen der Ports. Bei den Signalen *In1* und *Out1* ist erkennbar, dass der zuvor über einen *Data Type Conversion-Block* eingestellte Festkommatentyp übernommen wurde. Wichtig ist außerdem, dass die Ein- und Ausgänge für das *Valid-Signal* (*In2* und *Out2*) den Datentyp *boolean* besitzen, da dies der AXI-Stream-Konvention entspricht. Für den Ausgang *Out1* kann über die Schaltfläche **Options** zusätzlich die Größe des Ausgangspuffers konfiguriert werden. Standardmäßig ist dieser auf 1024 Bits eingestellt. Die Größe des Ausgangspuffers wurde auf 2^{20} eingestellt. Diese definierte Ausgangspuffergröße bestimmt die maximale Datenmenge, die pro Übertragung an den Filter gesendet werden kann. Diese Größe darf später weder unter- noch überschritten werden. Wird der Ausgangspuffer zu klein gewählt, muss das Eingangssignal in viele einzelne Übertragungen aufgeteilt werden. Da sich der IIR-Filter bei jeder Übertragung erneut einschwingt und keine Zustände zwischen den Übertragungen erhalten bleiben, kann es zu hörbaren Beeinträchtigungen kommen. Dieses wiederholte Einschwingen verstärkt Hintergrundrauschen und numerische Störungen, was sich in schwankender Lautstärke und zunehmendem Rauschen bemerkbar macht.

Der Haken bei **Generate default AXI4 slave interface** sollte entfernt werden, sofern er gesetzt ist. Andernfalls wird automatisch eine AXI4-Lite-Schnittstelle erzeugt. Diese wird üblicherweise

für die Registersteuerung verwendet. Da die hier entwickelte IP jedoch für einen kontinuierlichen Datenstrom ausgelegt ist, ist eine Steuer-Schnittstelle nicht erforderlich. Sie würde das Design unnötig verkomplizieren.

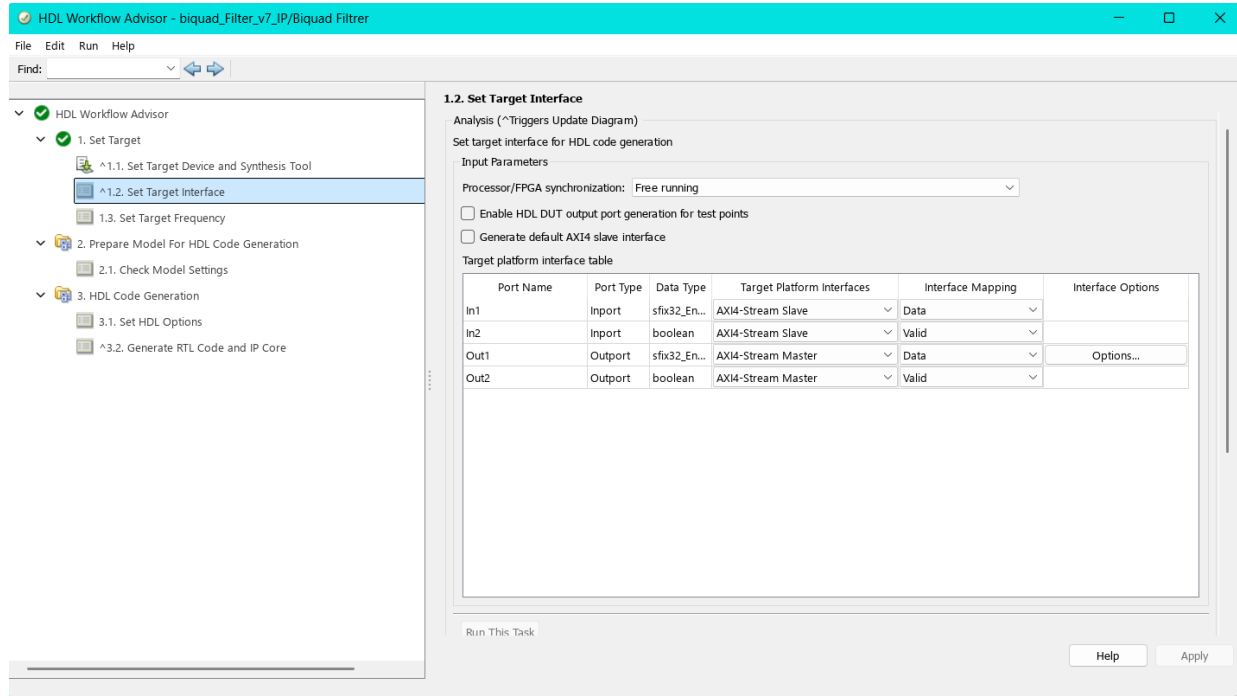


Abbildung 0.5: HDL Workflow: Target Interface

1.3 Set Target Frequency

An dieser Stelle wird die Zielfrequenz der zu generierenden IP festgelegt. Ursprünglich war diese auf 100 MHz gesetzt. Da jedoch bei der späteren Bitstream-Erzeugung die Timing-Anforderungen nicht erfüllt werden konnten, wurde die Frequenz auf 50 MHz halbiert. Diese reduzierte Taktfrequenz zeigte im Design stabile Timing-Ergebnisse und wurde daher für die weitere Implementierung beibehalten.

2.1 Check Model Settings

In diesem Schritt werden die verwendeten Simulink-Blöcke daraufhin überprüft, ob sie für die HDL-Codegenerierung geeignet sind. Zusätzlich kann ein **Industry Standard Check** durchgeführt werden, der das Modell hinsichtlich Industriestandards bewertet. Dabei werden häufig Warnungen ausgegeben, insbesondere bezüglich der Namenskonventionen, wenn diese nicht den typischen Industriestandards entsprechen. Für die hier verfolgte Anwendung ist es nicht erforderlich, den Industry Standard Check zu aktivieren, und es wird empfohlen, diesen nicht auszuführen, um unnötige Warnmeldungen zu vermeiden.

3.1 Set HDL Options

Hier werden die Einstellungen für die Codegenerierung getroffen. Im Bereich **Optimization/General** wurden die Einstellungen größtenteils unverändert übernommen. Dabei ist es jedoch wichtig darauf zu achten, dass die Option *Enable-based constraints* deaktiviert ist. Diese Funktion erzeugt zusätzliche Timing-Beschränkungen auf Basis von *Enable-Signalen*, was insbesondere bei einfacheren Designs oder kontinuierlich laufenden IPs zu unerwünschten Timing-Problemen führen kann.

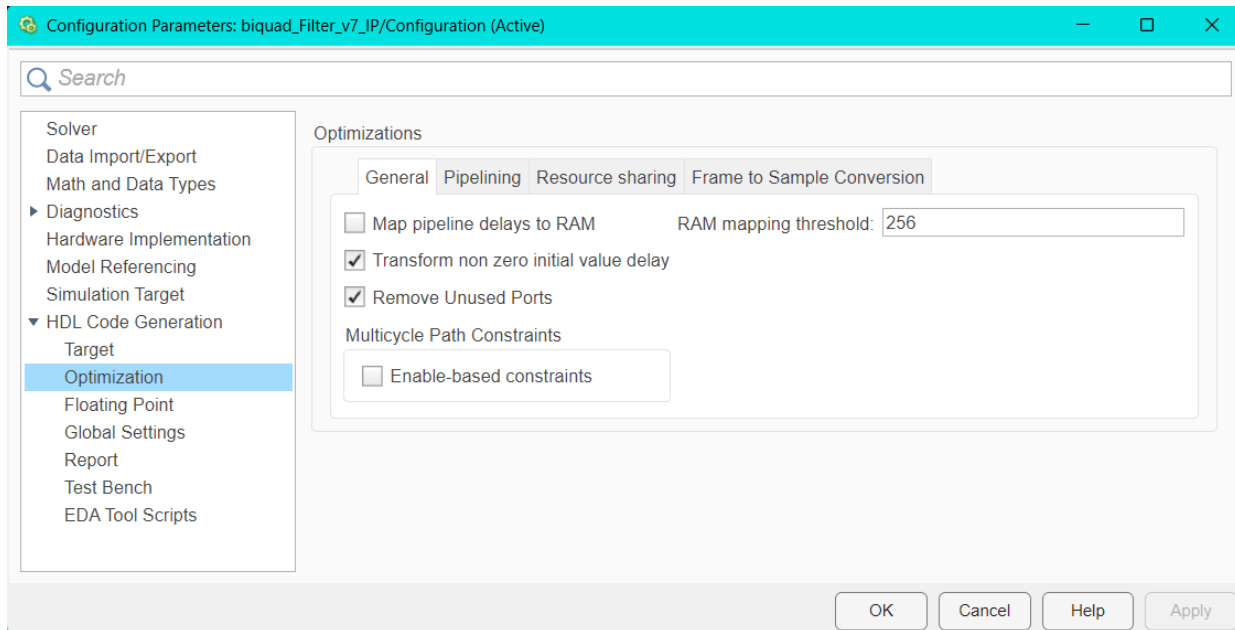


Abbildung 0.6: HDL Workflow: HDL Options/ Optimization/ General

Im Abschnitt **Optimization/Pipelining** blieben die Voreinstellungen ebenfalls weitgehend bestehen. Die Option *Adaptive Pipelining* kann dabei aktiviert werden. Diese Einstellung ermöglicht normalerweise eine automatische Optimierung der Pipeline-Struktur im Datenpfad. Da jedoch ein fertiger Block aus der DSP HDL Toolbox verwendet wurde, der intern bereits gepipelined ist, kann diese Option in diesem Fall auch deaktiviert bleiben. Während der Entwicklung wurde Adaptive Pipelining zunächst genutzt, um eine direkte Nachbildung der Direct-Form-Struktur zu pipelinen. Diese automatische Pipelining-Methode reichte jedoch nicht aus, um das Modell ausreichend zu optimieren.

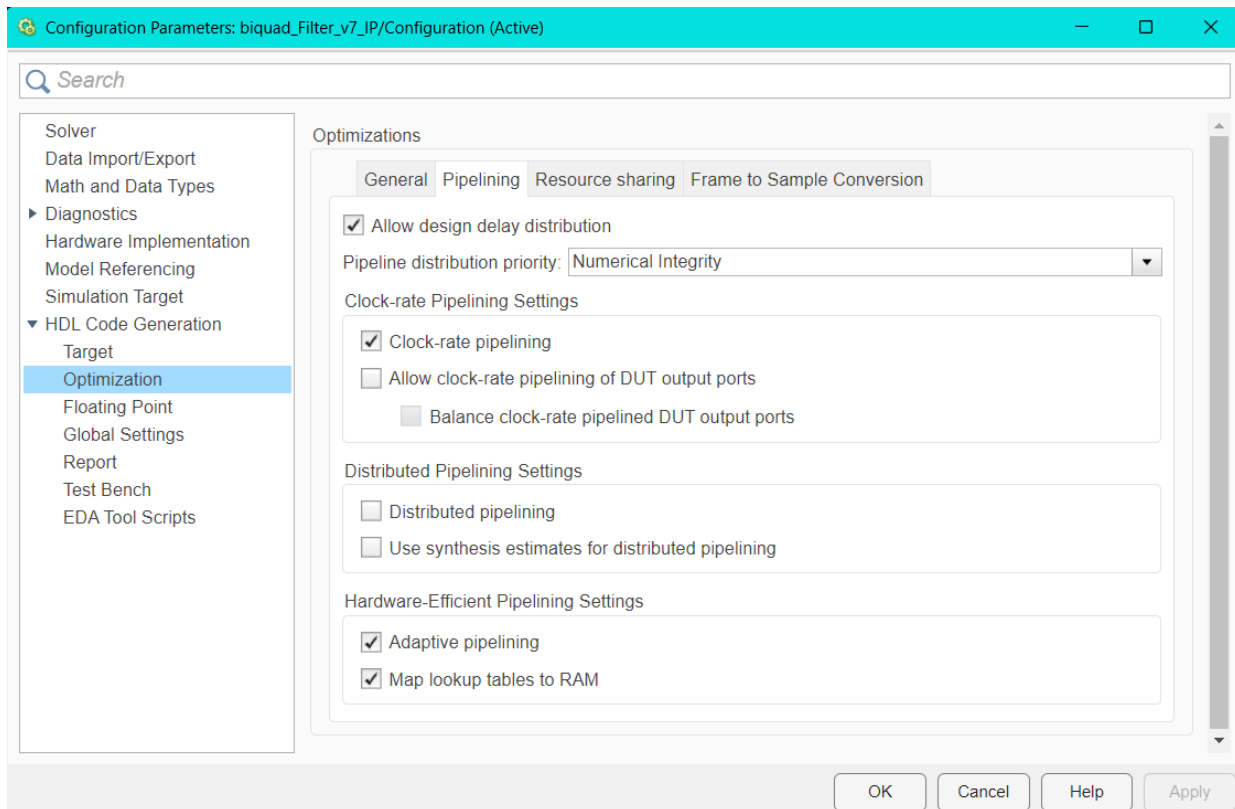


Abbildung 0.7: HDL Workflow: HDL Options/ Optimization/ Pipelining

Die **Global Settings** wurden größtenteils automatisch gesetzt. Dennoch sollte unbedingt geprüft werden, dass der *Reset type* auf *Synchronous* und der *Reset asserted level* auf *Active-high* eingestellt ist. Eine inkorrekte Reset-Konfiguration kann andernfalls beim Generieren der IP zu Warnungen oder Fehlern im Abschlussbericht führen.

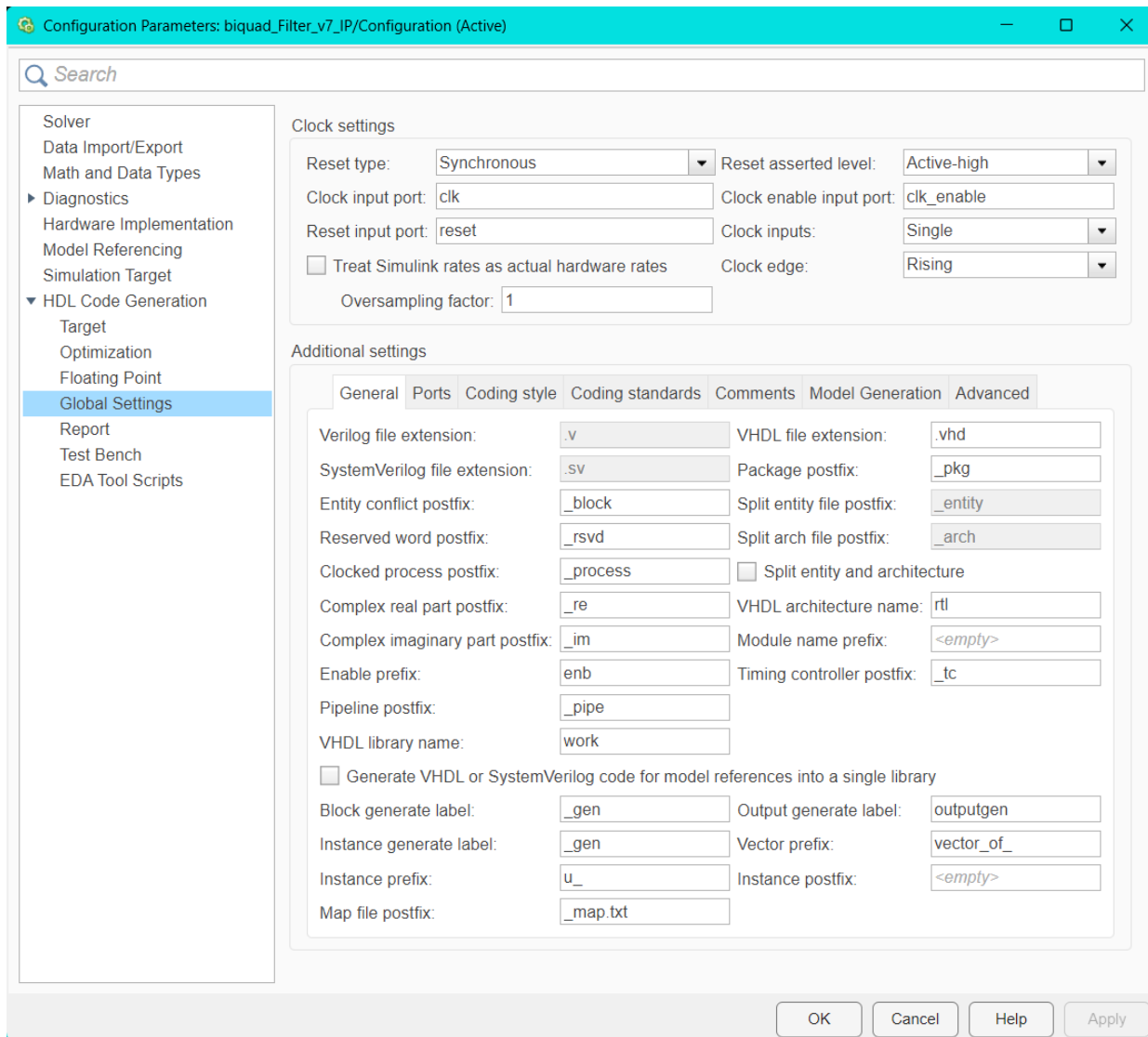


Abbildung 0.8: HDL Workflow: HDL Options/ Global Settings

3.2 Generate RTL Code and IP Core

Hier lässt sich der zu generierende IP-Core benennen und eine Versionsnummer vergeben. Für die IP sollte ein aussagekräftiger Name gewählt werden, damit sie später in Vivado leicht auffindbar ist.

Um die IP zu generieren und alle vorgenommenen Einstellungen anzuwenden und zu überprüfen, klickt man mit der rechten Maustaste auf den Schritt **3.2 Generate RTL Code and IP Core** und wählt *Run to Selected Task* aus. Dabei werden alle vorangehenden Schritte automatisch durchlaufen. Sollte ein Fehler auftreten oder eine Prüfung fehlschlagen, wird der Vorgang an der entsprechenden Stelle abgebrochen.

Nach der Generierung wird der IP-Core erstellt und ein Bericht geöffnet. Dieser enthält sowohl mögliche Warnungen als auch den erzeugten HDL-Code. Zusätzlich bietet der vollständige Report einen Überblick über die Ressourcennutzung, darunter beispielsweise die Anzahl verwendeter LUTs, Flip-Flops oder Multiplikatoren.

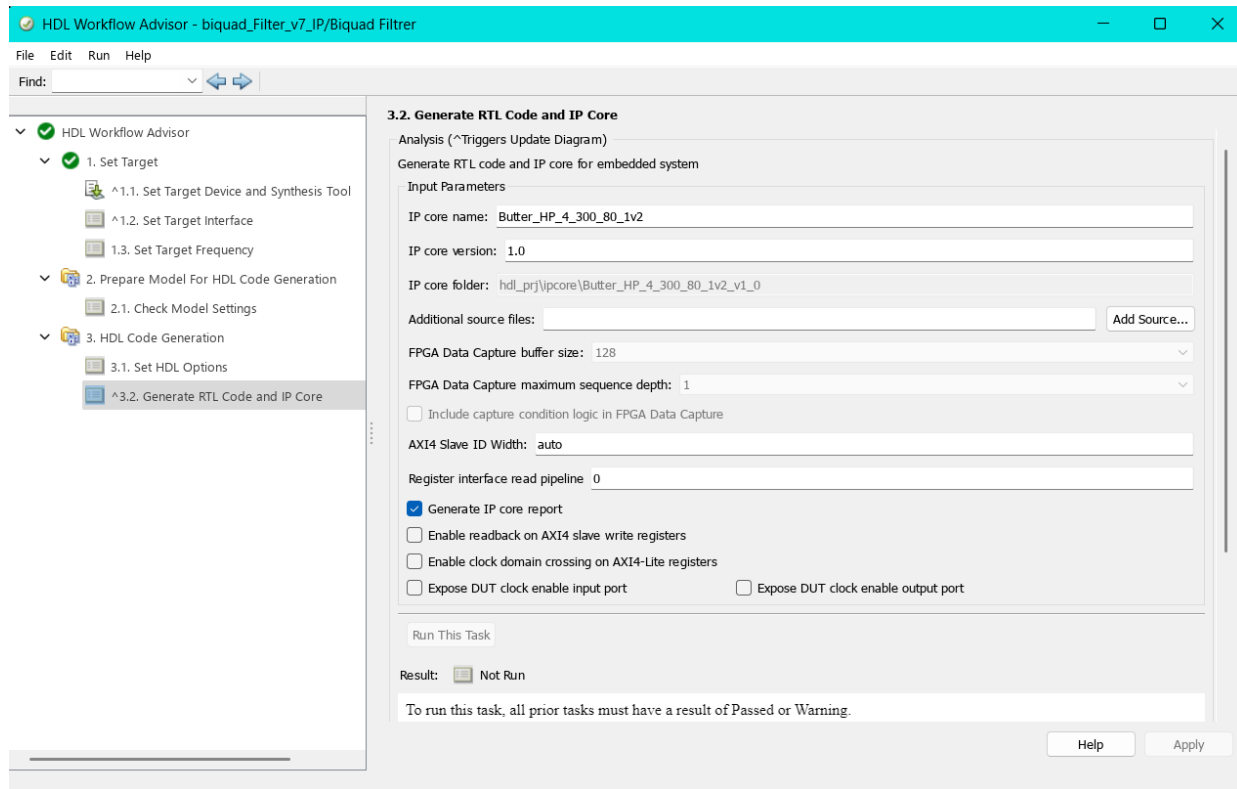


Abbildung 0.9: HDL Workflow: Generate RTL Code and IP Core

Für jeden Filter wird eine eigene IP generiert. Dabei wird jeweils dasselbe Simulink-Modell verwendet, lediglich die Koeffizienten werden angepasst.

Design in Vivado

Für die Aufnahme und Wiedergabe von Audio über das Pynq-Board wurde prinzipiell der Audioanteil aus dem Base-Overlay übernommen und die nicht benötigten Elemente entfernt. Die Einstellungen der Blöcke wurden ebenfalls übernommen. Die Constraints wurden so angepasst, dass die Audiofunktionalität erhalten blieb. Die Constraints weisen unter anderem den Benötigten internen PINs Namen zu, damit diese besser nachvollzogen werden können. Es ist notwendig, dass die Namen an dem Ein- und Ausgängen aus den Constraints übernommen werden. Für den Betrieb des Audio-Codes auf dem Pynq-Z2 sind spezielle IP-Cores von Pynq notwendig. Diese IPs sowie auch die Constraints befinden sich im [PYNQ GitHub Repository](#) und müssen in das Projekt

eingebunden werden. Vorgefertigte sowie auch eigens erstellte IP-Cores lassen sich in Vivado unter *Project Manager/Settings/IP/Repository* in das Projekt einbinden.

Teile des Designs orientieren sich an den offiziellen Tutorials von **Cathal McCabe**, die im AMD PYNQ-Forum verfügbar sind. Besonders hilfreich waren die Anleitungen [Creating a new hardware design for PYNQ](#) und [PYNQ DMA](#). Teile dieser Tutorials, insbesondere grundlegende Strukturkomponenten, wurden angepasst übernommen.

Audio Codec Controller

Der *audio_codec_controller* ist eine eigene IP von PYNQ und erzeugt die vom Audio-Codec benötigten Steuersignale sowie den *I²S* Datenstrom. Während im ursprünglichen Overlay ein *Clocking Wizard* eingesetzt wird, um aus dem 100 MHz-Systemtakt ein 10 MHz-Taktsignal zu erzeugen, stellt die überarbeitete Variante dieses 10 MHz-Signal direkt aus dem Processing System (PS) bereit, inklusive eines synchronisierten Reset-Signals. Diese Vorgehensweise hat den Vorteil, dass keine zusätzlichen Clock-Domain-Warnungen im Vivado auftreten.

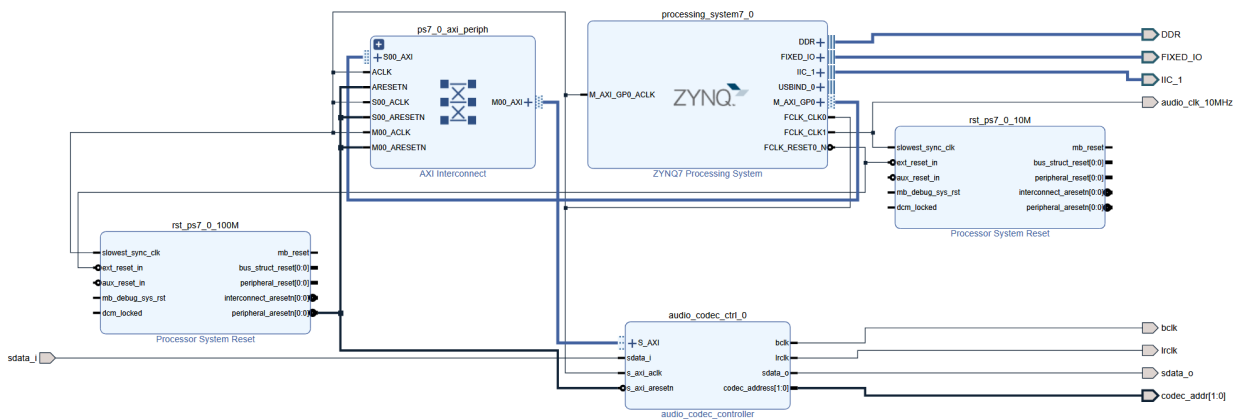


Abbildung 0.10: Reference Design

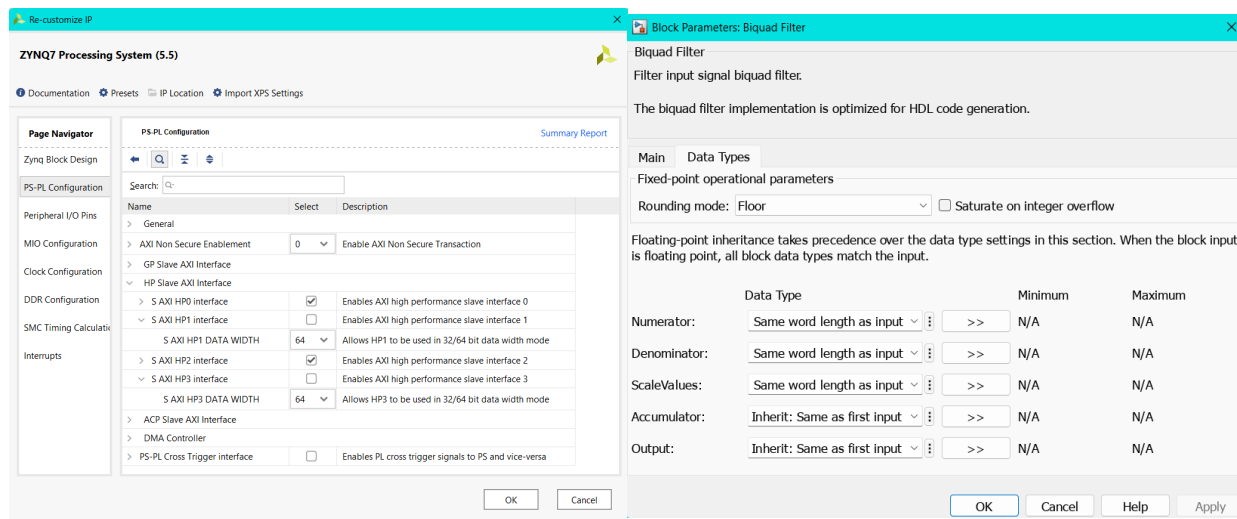
Processing System (PS)

Da das Design später über ein Jupyter Notebook gesteuert werden soll, wie es für die PYNQ-Plattform typisch ist, reicht eine reine RTL-Implementierung innerhalb der programmierbaren Logik nicht aus. Stattdessen wird das ZYNQ Processing System benötigt, das die Schnittstelle zwischen der Software-Ebene (Python/Jupyter) und der Hardware-Ebene (PL) bildet. Über das Processing System (PS) werden alle zentralen Taktsignale für das Design bereitgestellt. Zudem übernimmt das PS die Steuerung des Audio-Codexs sowie des AXI Direct Memory Access (AXI-DMA), über den die Filter mit dem PS verbunden sind. Die AXI-DMA-Verbindung ermöglicht einen Datenaustausch zwischen dem Arbeitsspeicher des PS und den in der programmierbaren Logik (PL) implementierten Filtern. Auf diese Weise kann das Jupyter Notebook über das PS

Daten an die Filter senden und deren Ausgaben empfangen. Damit der PS die korrekten Takt- und Schnittstellensignale erzeugt, muss er im Vivado-Design entsprechend konfiguriert werden.

PS-PL Configuration

Um Daten über die AXI-DMA-Schnittstellen an die Filter übergeben zu können, werden zusätzliche Anschlüsse am Processing System (PS) benötigt. Der PS besitzt intern zwei physische Zugänge zum DRAM, wobei sich jeweils zwei Ports (HP0/1 und HP2/3) einen gemeinsamen Zugriff teilen. Für dieses Design werden lediglich zwei High-Performance-Ports benötigt, weshalb HP0 und HP2 aktiviert werden. Die Datenbreite (Data Width) kann auf dem Standardwert von 64 Bit belassen werden.



(a) Vivado:PS Settings/ PS-PL-Configuration

(b) Vivado:PS Settings/ Clock Configuration

Clock Configuration

Alle benötigten Takt-Signale werden direkt im Processing System (PS) generiert und den jeweiligen Komponenten zugewiesen. Der Audio-Codec benötigt ein 10 MHz Taktsignal, während der Audio Codec Controller ein 100 MHz Taktsignal erfordert. Andere Frequenzen werden von dieser IP nicht unterstützt, was in Vivado durch eine entsprechende Warnung angezeigt wird. Die Filter-IP-Cores wurden auf 50 MHz ausgelegt und benötigen daher ebenfalls ein passendes 50 MHz-Taktsignal.

Peripheral I/O Pins

Für die Kommunikation mit dem Audio Codec Controller wird eine I^2C -Verbindung benötigt. Diese Schnittstelle lässt sich in der PS-Konfiguration unter *Peripheral I/O Pins* aktivieren, indem der entsprechende Haken gesetzt wird. In diesem Fall wurde $I^2C 1$ ausgewählt.

AXI Direct Memory Access

Die Anbindung der Filter-IP-Cores an das Processing System (PS) erfolgt über einen AXI Direct Memory Access (AXI-DMA). Dieser ermöglicht Daten direkt zwischen dem Speicher des Processing Systems (PS) und der programmierbaren Logik (PL) zu übertragen, ohne dass der Prozessor selbst aktiv Daten verschieben muss. Da im Design vier Filter parallel verwendet werden, werden auch vier AXI-DMA-Instanzen benötigt, die alle identisch konfiguriert werden. Eine konsistente Namenskonvention ist hierbei praktisch, da im Jupyter Notebook später über diese Namen auf die jeweiligen DMA-Schnittstellen zugegriffen wird.

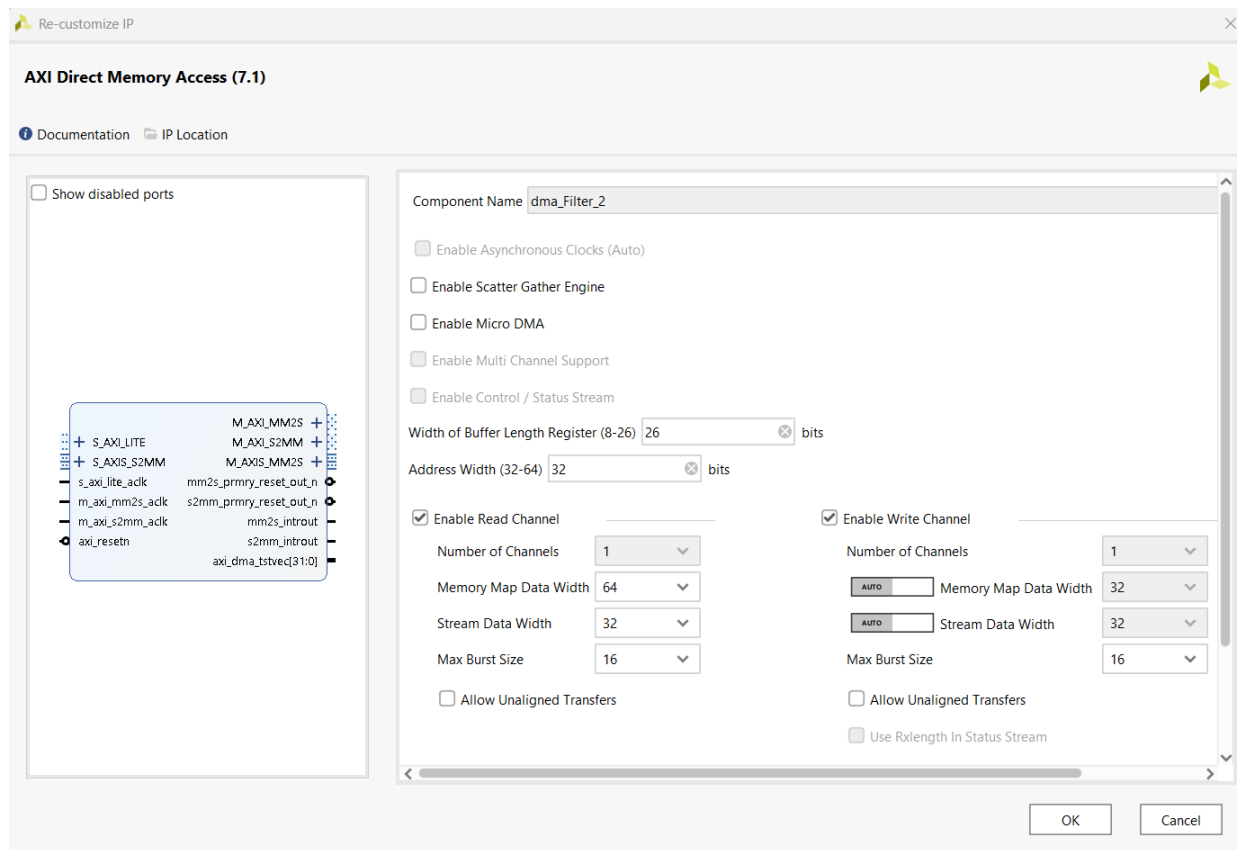


Abbildung 0.12: AXI-DMA Config

Bei der Konfiguration ist besonders darauf zu achten, dass sowohl *Enable Scatter Gather Engine* als auch *Enable Micro DMA* deaktiviert werden, da diese Funktionen in diesem Design nicht benötigt werden. *Allow unaligned transfers* sollte ebenfalls deaktiviert bleiben, da diese Funktion auf dem Pynq nicht unterstützt wird. Die *Width of Buffer Length Register* wird auf den maximalen Wert von 26 gesetzt. Auch wenn dieser Wert die Anforderungen der Filter übersteigt, wurde er aus Gründen der Robustheit und zur Vermeidung zukünftiger Einschränkungen bewusst beibehalten. Ein großer Puffer verhindert zudem, dass Fehler fälschlicherweise dem DMA zugeordnet werden, obwohl sie tatsächlich im Filter auftreten.

Die *Address Width* bleibt auf dem Standardwert von 32 Bit. Die *Stream Data Width* wird entsprechend der Konfiguration der *HP0/HP2-Ports* des PS auf 64 Bit gesetzt. Auch hier gilt Lieber großzügig dimensionieren als zu knapp, was ebenfalls aus der Entwicklungsphase übernommen wurde.

Die *Max Burst Size* wird auf 16 Bit gesetzt, obwohl die Burst-Funktion im eigentlichen Betrieb nicht verwendet wird. Für den *Write Channel* wird die Einstellung auf *Auto* belassen, wodurch automatisch die Parameter des Read-Channels übernommen werden.

Verbindungen im Blockdesign

Vivado bietet die Möglichkeit, Blöcke automatisch anhand von vorgeschlagenen Verbindungen miteinander zu verbinden. Bei standardisierten Schnittstellen sind diese Vorschläge in der Regel korrekt und können den Designprozess beschleunigen. Dennoch sollte stets überprüft werden, ob die vorgeschlagene Verbindung tatsächlich der gewünschten entspricht. Bei Verwendung dieser Funktion fügt Vivado oft automatisch unterstützende Blöcke wie beispielsweise einen *AXI Interconnect* hinzu, was zur besseren Übersichtlichkeit und Strukturierung des Designs beiträgt.

Eine Ausnahme bilden jedoch die Ein- und Ausgänge, die über die *Constraints-Datei (XDC)* definiert sind. Für diese Signale funktioniert die automatische Verbindung nur eingeschränkt oder fehlerhaft. Daher wird empfohlen, diese Verbindungen manuell vorzunehmen, um Fehler im späteren Designverlauf zu vermeiden.

S_AXI (S_AXI_LITE)

Dabei handelt es sich um die *AXI-Lite-Schnittstelle*, die für die Registersteuerung benötigt wird. Sowohl der *Audio-Codec-Controller* als auch die *AXI-DMA-Module* besitzen jeweils eine solche Verbindung. Diese werden allerdings auf unterschiedliche Clock-Frequenzen eingestellt. Der *Audio Codec Controller* wird auf 100MHz eingestellt und der *Axi-DMA* auf 50MHz.

High performace slave interface

Dies sind die Verbindungen an den *HP0/2-Ports* des Processing Systems (PS), über die die Daten zwischen dem BRAM und dem AXI-DMA übertragen werden. Diese Anschlüsse werden ausschließlich für die AXI-DMA-Kommunikation genutzt. Der *M_AXI_MM2S*-Port ist der *Read Channel*, und *M_AXI_S2MM* ist der *Write Channel*. Da die AXI-DMA-Module als Master fungieren, sind sie in der Lage, sowohl Lese- als auch Schreibzugriffe durchzuführen. Der PS übernimmt hierbei die Slave-Rolle. Diese Verbindungen werden bei allen vier AXI-DMA-Instanzen auf die gleiche Weise hergestellt. An den *HP0* wird der *M_AXI_MM2S* angeschlossen und an den *HP2* der *M_AXI_S2MM*, beide auf 50MHz.

AXI-Stream

Die Filter sowie die AXI-DMA-Module besitzen jeweils einen *S_AXIS_2SMM* bzw. *AXI4_Stream-Slave* (Slave) und einen *M_AXIS_MM2S* bzw. *AXI4_Stream-Master* (Master) Anschluss. Da es sich beim AXI-Stream um eine gerichtete Verbindung handelt, ist es entscheidend, dass immer ein Master-Port mit einem Slave-Port verbunden wird. Diese Verbindungen müssen manuell vorgenommen werden, da Vivado diese Zuordnungen nicht automatisch erkennen kann. Nachdem die Datenpfade korrekt verbunden wurden, schlägt Vivado in der Regel automatisch eine Verbindung für die zugehörigen Taktsignale und Reset-Signale vor. Wichtig ist dabei, das richtige Taktsignal auszuwählen, in diesem Fall 50 MHz, da die Filter in dieser Taktfrequenz ausgelegt sind.

Vollendetes Design

Sind alle Verbindungen korrekt vorgenommen, ergibt sich folgendes Blockdesign:

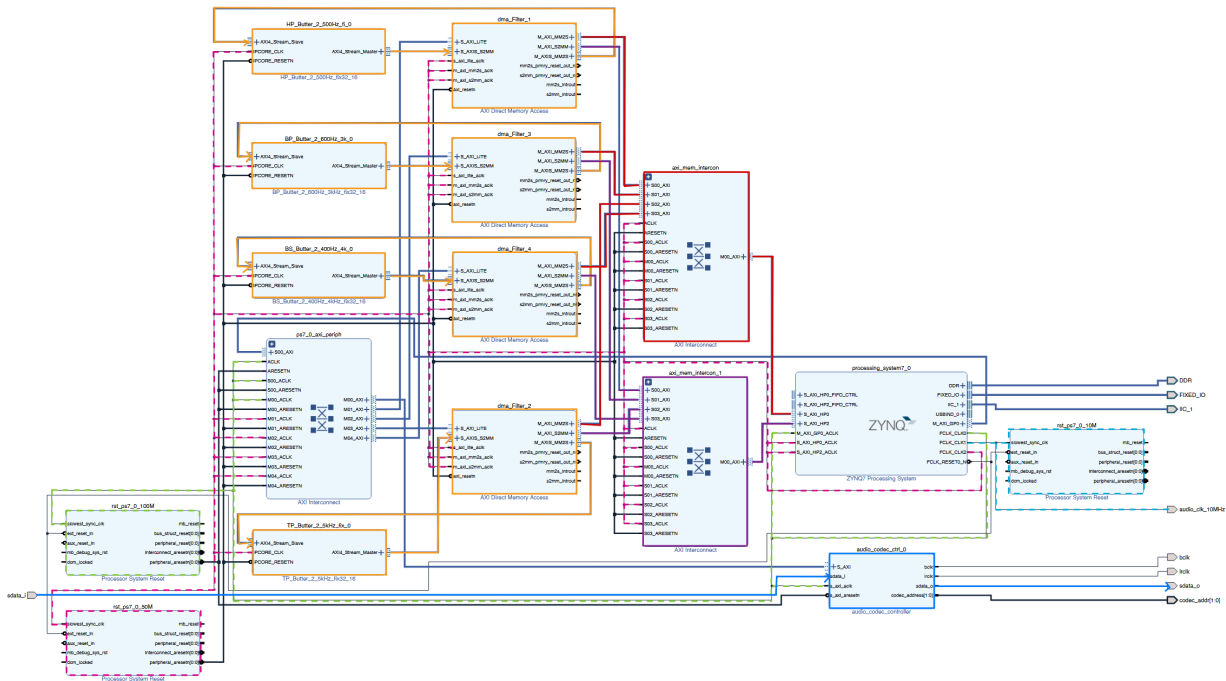


Abbildung 0.13: Blockdesign

Die Clock-Signale sowie die wichtigsten Datenpfade wurden zusätzlich farblich hervorgehoben, um die Struktur übersichtlicher darzustellen. Nach erfolgreicher Validierung des Designs kann im nächsten Schritt der Bitstream generiert werden.

Am Ende besteht ein vollständiges Overlay aus zwei Dateien. Der generierten Bitstream-Datei (.bit) und der zugehörigen Hardware-Handoff-Datei (.hwh), welche die notwendigen Metadaten

über das Design beinhaltet. Damit das Overlay von PYNQ korrekt erkannt und geladen werden kann, müssen beide Dateien denselben Basisnamen tragen mit Ausnahme der Dateiendung.

Funktionsweise des generierten Filter-Codes

Im Folgenden wird der generierte VHDL-Code der Filter näher erläutert. Die Beschreibung bezieht sich exemplarisch auf den Hochpassfilter. Da jedoch alle Filter nach dem gleichen Verfahren erzeugt wurden, lassen sich die Ausführungen sinngemäß auf sämtliche Filter übertragen. Die Modulnamen wurden automatisch vom HDL-Coder generiert und unverändert übernommen. Der resultierende VHDL-Code zur Beschreibung der Filter-IP-Core ist aufgrund der zahlreichen verschachtelten Strukturen nur schwer überschaubar. Daher wird an dieser Stelle auf konkrete Codeausschnitte verzichtet. Stattdessen erfolgt eine allgemeine Darstellung des strukturellen Aufbaus und der funktionalen Gliederung der generierten Dateien. Hinzu kommt, dass der Code bei der Generierung verschiedene Optimierungsverfahren, insbesondere Pipelining, durchlaufen hat. Dadurch ist die ursprüngliche Filterstruktur im finalen VHDL-Code kaum noch erkennbar.

Struktur des generierten Codes

Das Top-Level-Modul bildet die oberste Instanz und integriert alle Funktionseinheiten. Die AXI4-Stream-Schnittstellen werden von zwei Modulen umgesetzt, einem AXI4-Stream Slave und einem Master. Der Slave empfängt die Eingangsdaten vom übergeordneten System und speichert sie zunächst in einem Eingangs-FIFO. Die eigentliche Signalverarbeitung findet im Design Under Test (DUT) statt. Hier werden die zwischengespeicherten Eingangsdaten durch die Biquad-Filtersektionen geleitet. Jede Sektion wird dabei durch ein separates Modul realisiert. Das DUT enthält ein Modul, das die gesamte Biquad-Filterstruktur beschreibt, die wiederum aus mehreren Sektionen bestehen kann. Die Daten werden Sektion für Sektion gefiltert, bis die gewünschte Filtercharakteristik erreicht ist.

Nach der Filterung werden die Ausgangsdaten in einem Ausgangs-FIFO zwischengespeichert, damit sie gepuffert zur Verfügung stehen. Ein spezieller FIFO sorgt für die korrekte Handhabung des TLAST-Signals, das im AXI4-Stream-Protokoll das Ende eines Frames oder Datenpakets signalisiert. Der AXI4-Stream Master gibt die gefilterten Daten schließlich zurück an das übergeordnete System. Ein zusätzliches Modul zur Reset-Synchronisation stellt sicher, dass ein globales Reset-Signal in allen Takt- und Logikbereichen zuverlässig übernommen wird.

Für den Datenverlauf bedeutet das, dass die Eingangsdaten über einen AXI4-Stream empfangen, gepuffert, durch den Biquad-Filter verarbeitet, erneut gepuffert und schließlich über AXI4-Stream ausgegeben werden. Der Datenfluss wird über Handshake-Signale gesteuert, um Datenverlust oder Überläufe zu vermeiden.

Umsetzung des Filters

Filter DUT

Wie bereits erwähnt, folgt der Code einer klaren Hierarchie. Das DUT übernimmt innerhalb dieses Designs mehrere Aufgaben. Es steuert, wann neue Daten aus dem Eingangs-FIFO gelesen werden und führt diese der Filter-Komponente zu. Nach der Filterung nimmt es die Ergebnisse entgegen, speichert sie im Ausgangs-FIFO und kümmert sich um alle notwendigen Steuersignale wie *Valid/Ready*-Signale und Handshake-Mechanismen. Zudem stellt es sicher, dass die Filter-Komponente korrekt in die übergeordnete Takt- und Reset-Logik eingebunden ist.

Filter-Wrapper

Innerhalb des DUT wird die Komponente **src_Biquad_Filter** instanziiert. Diese Komponente fungiert als Wrapper und kapselt die eigentliche Filterkette. Sie nimmt die Eingangssignale entgegen, leitet sie unverändert weiter und gibt die Ergebnisse der Filterkette zurück an das DUT. Zudem stellt der Wrapper sicher, dass die Port-Zuordnung und Taktanbindung korrekt sind und das *clk_enable-Signal* richtig durchgereicht wird. Auf diese Weise wird die gesamte Filterlogik mit den Biquad-Sektionen innerhalb des Wrappers gebündelt. Es ist anzumerken, dass in der tatsächlichen Implementierung kein *clk_enable-Signal* aktiv genutzt wird, dennoch wurde es automatisch mitgeneriert.

Die Filterlogik

Wie bereits erwähnt, wird die Filterlogik exemplarisch am Hochpass-Filter erläutert. Es kann dabei zu minimalen Abweichungen bei anderen Filtern kommen, die grundsätzliche Funktionsweise ist jedoch bei allen Filtern vergleichbar.

Innerhalb des Wrappers wird eine gleichnamige Komponente instanziiert, die hierarchisch unterhalb des Wrappers angesiedelt ist. Hier befindet sich die eigentliche Filterlogik. Die Eingangsdaten (*dataIn*) werden zunächst in ein vorzeichenbehaftetes Signal (*dataIn_signed*) umgewandelt und im Register *sec0reg* zwischengespeichert. In der ersten Shifting- und Multiplikationsstufe (*sec0*) wird der Eingangswert durch Konkatenation mit Nullen nach links verschoben, um die gewünschte Fixpunkt-Skalierung zu erreichen. Das Ergebnis wird im Register *sec0mulreg* abgelegt, der relevante Bitbereich extrahiert (*sec0dte*) und in *sec0out* zwischengespeichert. Von dort wird das Signal an die erste Biquad-Sektion weitergegeben.

Parallel dazu wird das Valid-Signal über eine Signalkette geführt, sodass die Gültigkeit der Daten in jeder Stufe erhalten bleibt. Die instanziierte Biquad-Sektion verarbeitet *sec0out* und liefert die gefilterten Daten (*sec1out*) sowie das zugehörige Valid-Signal (*sec1validout*). Anschließend folgt eine zweite Shifting- und Multiplikationsstufe (*sec1*), in der *sec1out* erneut skaliert und vorbereitet wird. Auch hier wird das Valid-Signal entsprechend weitergereicht. Für einen Filter zweiter Ordnung wird diese Sektion nur einmal aufgerufen, bei höheren Ordnungen werden mehrere Sektionen kaskadiert. Die Ausgabe wird konditional gesteuert. Wenn das Valid-Signal gültig ist, wird das

gefilterte Ergebnis an *dataOut* ausgegeben. Ein synchronisiertes Valid-Signal (*validOut*) zeigt an, wann die Daten gültig sind.

Filter-Sektion

Die Filter-Sektion bildet die eigentliche Biquad-Filterstruktur und enthält die entsprechenden Koeffizienten. Die Biquad-Sektion ist in transponierter Direct Form II umgesetzt. Sie erhält ein Eingangssample (*dataIn*) mit zugehörigem Gültigkeitssignal (*validIn*) und liefert das gefilterte Sample samt gültigem Ausgangssignal zurück.

Zunächst wird das Eingangssignal gepuffert. Anschließend wird es in drei Numerator-Zweigen parallel mit festen Koeffizienten multipliziert. Jeder Pfad besteht aus einem Vor-Register, dem Multiplizierer und einem Nach-Register. Die Denominator-Seite arbeitet mit zwei internen Zuständen (*state1* und *state2*), die über Multiplikationen und Addierer-Ketten rekursiv zurückgeführt werden. Diese Rückkopplung bildet den IIR-Charakter der Filterung.

Die kombinierte Summe wird auf den gewünschten Fixpunktbereich begrenzt und ausgegeben. Ein mehrstufiges Delay-Register sorgt dafür, dass das Ausgangssignal und das Valid-Signal zeitlich synchron anliegen. Zusammengefasst wird das Eingangssignal gepuffert, in drei parallelen Numerator-Zweigen mit festen Koeffizienten verarbeitet und mit den Rückkopplungswerten kombiniert. Die resultierende Summe wird skaliert und als gefiltertes Signal ausgegeben, während das Valid-Signal die zeitliche Korrektheit sicherstellt.

Echtzeitfilterung

Ursprünglich wurden die Filter als AXI4-Stream-fähige IP-Cores umgesetzt, um sie universell innerhalb von Vivado einsetzen zu können. Die bisherige Anwendung erfolgte über die Anbindung an einen AXI-DMA, um digitale Audioquellen zu filtern. Geplant war jedoch, denselben Filter (und gegebenenfalls weitere) auch für die Echtzeitverarbeitung analoger Audioquellen zu verwenden. Hierzu sollten die in Vivado verfügbaren LogiCORE- I^2S -Transmitter und -Receiver IPs in Kombination mit dem Audio-Codec eingesetzt werden.

In der vorgesehenen Konfiguration arbeitet der Audio-Codec-Controller als Master, d. h. er gibt die Taktsignale *bclk* und *lrclk* für die I^2S -Kommunikation vor. Der I^2S -Transmitter und -Receiver müssten in diesem Fall als Slaves fungieren. Genau hier trat das erste Problem auf. [Laut offizieller Dokumentation](#) unterstützen die I^2S IP-Cores keinen Slave-Modus, sondern sind ausschließlich für den Master-Betrieb vorgesehen. Da der Audio-Codec-Controller fest als Master konfiguriert ist, ergab sich ein Konflikt. Mehrere Master in einer I^2S -Verbindung würden unweigerlich zu Timing-Problemen führen.

Trotz dieser Einschränkung gelang es, durch gezielte Block-Property-Einstellungen in Vivado sowohl den I^2S -Receiver als auch den Transmitter im Slave-Modus zu betreiben, entgegen der offiziellen Angaben. Der Audio-Codec-Controller blieb dabei weiterhin Taktgeber. Die Initialisierung der I^2S -IP-Cores erfolgte über ein Jupyter Notebook, in dem die Register gemäß Datenblatt beschrieben wurden. Während für den Receiver die Samplerate und das Enable-Register gesetzt werden mussten, reichte beim Transmitter das Aktivieren über das Enable-Register aus.

Funktionstest & Filtereinbindung

In einem ersten Test wurde der Receiver direkt mit dem Transmitter über AXI-Stream verbunden. Diese Konfiguration funktionierte einwandfrei. Das Audiosignal wurde korrekt wiedergegeben, unabhängig davon, ob Receiver und Transmitter vor oder nach dem Codec-Controller eingebunden waren. Anschließend wurde der eigene Biquad-Filter-IP-Core zwischen Receiver und Transmitter geschaltet. Zwar blieb das Signal grundsätzlich vorhanden, war jedoch sehr leise und klang unverändert, eine effektive Filterung konnte nicht festgestellt werden.

Zur Fehlereingrenzung wurde eine vereinfachte Filter-IP-Core entwickelt, die nur als Bypass ohne jegliche Signalverarbeitung fungierte. Doch auch in diesem Fall blieb das Ausgangssignal stark gedämpft. Die Ursache lag nicht in der Filterlogik selbst.

Weitere Tests bestätigten, dass der vom HDL Coder erzeugte AXI-Datenpfad, bestehend aus TDATA, TVALID und TREADY, korrekt generiert wurden. Verschiedene Datenformate (signed/unsigned), Skalierungsfaktoren, Gain-Blöcke und Bit-Shifts hatten keinen nennenswerten

Einfluss auf die Lautstärke. Im Gegensatz dazu führte der Einsatz eines [AXI4-Stream Data FIFO](#) zwischen Receiver und Transmitter zu einer fehlerfreien, normal lauten Wiedergabe. Dies ließ den Verdacht aufkommen, dass der FIFO intern ein AXI-konformes Handshake-Verhalten sicherstellt, das von der eigenen IP nicht vollständig erfüllt wird.

Eine selbst erstellte IP mit identischer Buffertiefe (1024 Bit) brachte allerdings keine Verbesserung. Auch das optionale AXI-Signal TID, das bei den I^2S -IP-Cores zur Kanalunterscheidung dient, konnte in Simulink weder direkt erzeugt noch verarbeitet werden. Selbst ein manuelles Durchreichen außerhalb der IP zeigte keine Wirkung. Ein zusätzlicher FIFO hinter der IP, als Versuch, den Handshake zu stabilisieren, blieb ebenso erfolglos. Auch die Kombination von FIFO vor und nach der IP brachte keine Verbesserung, wodurch der Handshake als Fehlerursache weitgehend ausgeschlossen werden konnte. Der HDL Workflow Advisor bestätigte, dass alle AXI-Signale korrekt zugewiesen wurden und sogar eine explizite Steuerlogik für TREADY generiert wurde.

Ursache und Bewertung

Eine eindeutige Ursache für das fehlerhafte Verhalten konnte nicht ermittelt werden. Die Vermutung liegt jedoch nahe, dass die minimale AXI-Stream Konfiguration der vom HDL Coder generierten IP nicht alle Protokollanforderungen erfüllt, die der I^2S -Transmitter erwartet. Besonders die fehlende Unterstützung für optionale Signale wie TID und eine nicht vollständige Flusskontrolle könnten das Verhalten erklären. Um dieses Problem zu beheben, wäre ein ausgereifteres und weiter ausgeführtes Design erforderlich, das gegebenenfalls zusätzliche Signale berücksichtigt. Ein solches Design müsste unter Umständen manuell in VHDL oder Verilog implementiert werden und würde tiefgehende Kenntnisse des AXI-Protokolls, detaillierte Timinganalysen sowie ergänzende Simulationsschritte außerhalb von MATLAB/Simulink erfordern. Aufgrund der begrenzten Projektzeit und dem Fokus auf die eigentliche Filterimplementierung wurde entschieden, die Echtzeit-Anbindung über I^2S Receiver/Transmitter nicht weiterzuverfolgen. Stattdessen wird der Filtereinsatz über AXI-DMA demonstriert, die bereits erfolgreich mit digitalen Quellen funktioniert.

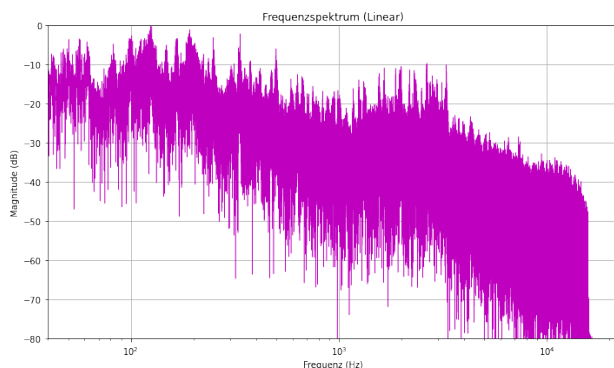
Analoge Audioquellen

Da die geplante Echtzeitfilterung analoger Signale nicht zeitnah umsetzbar war, wurde das Design zur Filterung digitaler Quellen entsprechend erweitert. Mithilfe des auf dem PYNQ-Z2-Board integrierten Audio-Codecs ist es nun möglich, analoge Audiosignale aufzunehmen und als Wav-Datei auf dem Board zu speichern. Diese digitale Datei kann anschließend vom bestehenden Design verarbeitet und gefiltert werden. Dadurch ist es trotz der Einschränkungen bei der Echtzeitverarbeitung möglich, analoge Audioquellen zu filtern, jedoch mit einer zwischengelagerten, nicht-echtzeitfähigen Verarbeitungskette.

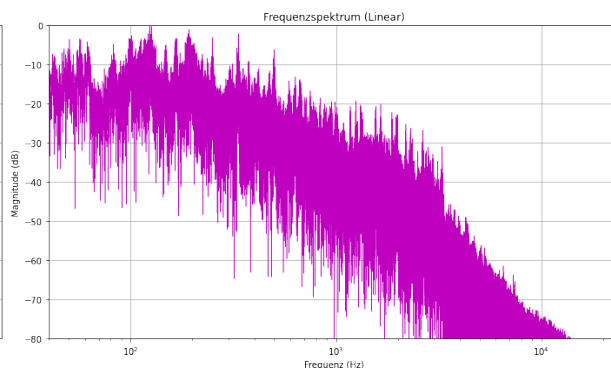
Aufbau und Funktion der Lehrdemonstration

Ein zentrales Element der Lehrdemonstration ist ein vorgefertigtes Jupyter-Notebook, mit dem sich die verschiedenen Filter einfach bedienen lassen. Innerhalb dieses Notebooks können analoge Audiosignale über die Eingänge des Boards aufgenommen und sowohl im Notebook selbst als auch über den analogen Audioausgang des PYNQ-Z2 wiedergegeben werden. Darüber hinaus besteht die Möglichkeit, sowohl die aufgenommenen Audiodaten als auch externe Audiodateien mit einem der vier implementierten Filter zu verarbeiten. Nach der Auswahl eines Filters wird das gefilterte Signal gespeichert und kann angehört oder weiter analysiert werden.

Da sowohl das Audiosignal vor als auch nach der Filterung gespeichert wird, besteht die Möglichkeit, beide Versionen direkt miteinander zu vergleichen. Neben dem reinen Abhören der Audiodateien kann zusätzlich ein definierter Ausschnitt des Frequenzspektrums vor und nach der Filterung visualisiert werden. Dies ermöglicht eine anschauliche Analyse der Signalveränderung im Frequenzbereich und macht die Wirkung des gewählten Filters unmittelbar sichtbar. Auf diese Weise wird die digitale Signalverarbeitung nicht nur hörbar, sondern auch grafisch nachvollziehbar.



(a) Frequenzspektrum vor dem Filtern



(b) Frequenzspektrum nach der Tiefpassfilterung

In den Abbildungen ist nicht das vollständige Frequenzspektrum dargestellt. Stattdessen wird lediglich ein Ausschnitt von den ersten 30 Sekunden des linken Audiokanals gezeigt. Dies wurde bewusst so gewählt, da eine vollständige Analyse des gesamten Signals rechnerisch zu aufwendig wäre, um sie im Jupyter Notebook durchzuführen. Die Darstellung erfolgt in normierter Form. Die Amplitudenwerte werden so skaliert, dass der jeweils höchste Wert bei 0 dB liegt. Dadurch kann es zwar zu Unterschieden in der absoluten Lautstärke zwischen verschiedenen Plots kommen, jedoch bleibt der relative Vergleich zwischen verschiedenen Frequenzverläufen jederzeit möglich.

Pynq Overlays

Zur Nutzung individuell entwickelter Hardwaredesigns auf dem PYNQ-Z2-Board werden Overlays verwendet. Diese bestehen aus einer Bitstream-Datei (.bit) und der zugehörigen Hardware Handoff-Datei (.hwh), die zuvor mit Vivado generiert werden. In einigen PYNQ-Dokumentationen wird teilweise auch auf .tcl-Dateien verwiesen. Im Rahmen der hier eingesetzten PYNQ-Version hat sich gezeigt, dass diese Datei nicht erforderlich ist. Das Overlay lässt sich vollständig mit der .bit- und der .hwh-Datei verwenden.

Die eigentliche Ansteuerung der Hardware erfolgt über das auf dem PYNQ-Board vorinstallierte Python-Modul [pynq](#). Dieses Modul stellt eine spezielle Bibliothek bereit, mit der sich aus Python heraus direkt auf die programmierbare Logik (PL) zugreifen lässt. Dadurch können AXI-DMA-Schnittstellen genutzt, Register beschrieben oder externe IP-Cores über vordefinierte Treiber angesprochen werden. Die Steuerung der Hardware erfolgt in Python über Jupyter Notebooks, die direkt auf dem PYNQ-Board ausgeführt werden. Der Zugriff auf die Notebooks erfolgt remote über Geräte im selben Netzwerk.

Zum einen wird das Modul [pynq.lib.dma](#) verwendet, das den Zugriff auf die AXI-DMA-Schnittstellen der programmierbaren Logik ermöglicht. Über dieses Modul können mit wenigen Python-Befehlen Daten an die Filter-IP-Cores gesendet und die verarbeiteten Daten wieder empfangen werden. Die Übertragung wird dabei automatisch vom Modul verwaltet, sodass keine manuelle Kontrolle über Adressen notwendig ist

Zum anderen kommt das Modul [pynq.lib.audio](#) zum Einsatz. Dieses Modul enthält den Treiber für den auf dem PYNQ-Z2-Board verbauten ADAU1761-Audiocodec und ermöglicht es, die analogen Audioein- und -ausgänge des Boards direkt aus Python heraus anzusteuern. Über den Treiber können zentrale Funktionen wie das Setzen der Abtastrate, das Aktivieren oder Deaktivieren von Ein- und Ausgängen gesteuert werden. Darüber hinaus bietet das Modul die Möglichkeit, Audiosignale über die analogen Eingänge aufzunehmen.

Beim Versuch, die Abtastrate von 48 kHz auf 44,1 kHz umzustellen, zeigte sich jedoch, dass die Aufnahmen trotz korrekter Konfiguration weiterhin mit 48 kHz durchgeführt wurden. Zwar wurde in der erzeugten WAV-Datei die Samplingrate ordnungsgemäß mit 44,1 kHz eingetragen, doch beim Abhören war ein deutlich tieferer Pitch zu erkennen, was ein Anzeichen dafür ist, dass die tatsächliche Aufnahmefrequenz höher lag als angegeben. Bei der direkten Wiedergabe über die Hardware blieb dieser Fehler zunächst unbemerkt. Sobald die Aufnahme über das Notebook abgespielt wurde, war der Pitch sofort hörbar. Auch externe WAV-Dateien mit 44,1 kHz klangen über den Hardware-Ausgang zu schnell, wenn dieser auf 44,1 kHz eingestellt wurde, was ebenfalls auf eine Diskrepanz zwischen eingestellter und tatsächlich verwendeter Abtastrate hinweist. Aus diesem Grund wurde das gesamte Filterdesign konsequent auf eine Samplingrate von 48 kHz ausgelegt. Um dennoch Audiodateien mit niedrigeren Abtastraten verarbeiten zu können, werden diese vor der Weiterverarbeitung entsprechend hochgesampelt.

Funktionsweise des Notebooks

Zur besseren Übersicht und vereinfachten Bedienung wurden alle wesentlichen Prozesse, wie Datenübertragung, Speichern und Visualisierung, in einer separaten Python-Datei als einheitliche Funktionssammlung zusammengeführt. Diese Struktur ermöglicht es, komplexe Abläufe mit nur wenigen, klaren Funktionsaufrufen direkt im Jupyter Notebook auszuführen.

Die wichtigste Rolle spielt dabei die Übertragung der Audiodaten an den DMA, der diese durch den Filter-IP-Core leitet und anschließend wieder zurückgibt. Die grundlegende Datenübertragung an den DMA erfolgt dabei relativ einfach und orientiert sich im Prinzip an dem Ablauf, wie er auch in den offiziellen PYNQ-Tutorials und der [PYNQ-Dokumentation](#) beschrieben ist.

Initialisierung des DMA-Kanals

Am Anfang werden alle notwendigen Bibliotheken importiert, einschließlich der PYNQ-spezifischen Module wie *Overlay*, *allocate* sowie die Treiberbibliothek *pynq.lib.audio*. Anschließend wird das entsprechende DMA-IP-Core aus dem zuvor geladenen Overlay angesprochen und in einer Variable gespeichert. Die DMA-Instanz verfügt über zwei Kanäle. Für jeden der vier Filter wird eine eigene DMA-Instanz erzeugt und jeweils den zugehörigen Variablen zugewiesen.

```
from pynq import Overlay
from pynq import allocate
from pynq.lib.audio import AudioADAU1761

ol = Overlay("Audio_quad_Filter_v7.bit")

dma_Filter_1 = ol.dma_Filter_1
dma_Filter_1_send = ol.dma_Filter_1.sendchannel
dma_Filter_1_recv = ol.dma_Filter_1.recvchannel
```

DMA Übertragung

Eine Zentrale Funktion ist die *Transmission(input_data, ip_buffer, dma, dma_send, dma_recv)*. Diese Funktion wird für jede Übertragung aufgerufen und übernimmt die Kernaufgaben für eine vollständige Übertragung.

Bevor die Audiodaten an die DMA-Schnittstelle übergeben werden können, müssen sie zunächst normiert und in ein kompatibles Datenformat umgewandelt werden. Dies ist notwendig, da die AXI-DMA-Schnittstelle nur mit bestimmten Datentypen arbeiten kann.

```
# Festlegen der Größen
buffer_size = int(ip_buffer)
input_data = FormatChange(input_data)
data_size = int(len(input_data))
```

Die DMA-Instanz verfügt über zwei Kanäle, den *sendchannel* zum Senden von Daten und den *recvchannel* zum Empfangen der verarbeiteten Daten. Diese beiden Kanäle werden für die gewählte DMA-Instanz initialisiert. Bevor die Audiodaten in den *input_buffer* geladen werden, wird dieser zunächst vollständig mit Nullwerten gepuffert. Der Hintergrund ist, dass beim Anlegen des Buffers mittels *allocate()* dieser zunächst undefinierte Werte enthält. Falls die zu übertragende Datenmenge kleiner als die definierte Buffergröße ist, bleiben einzelne Felder leer. Der AXI-DMA prüft jedoch bei jedem Transfer, ob gültige Daten im Puffer vorhanden sind. Sind nicht alle Speicherbereiche korrekt befüllt, kann der DMA in einen Fehlerzustand übergehen. In diesem Fall wird der gesamte Datenpfad blockiert, und es sind keine weiteren Übertragungen möglich, bis ein manueller Reset durchgeführt wird. Um diesem vorzubeugen, wird der *input_buffer* vor dem eigentlichen Datentransfer vollständig mit Nullen überschrieben. Anschließend werden die eigentlichen Nutzdaten an den Anfang des Buffers geschrieben. Nach der Übertragung wird der Puffer anhand der bekannten Datenlänge wieder korrekt zurückgeschnitten.

```
# Padding
pad = np.zeros(ip_buffer)
pad_frame = FormatChange(pad)

# Leere Buffer
input_buffer = allocate(shape=(buffer_size,), dtype=np.uint32)
output_buffer = allocate(shape=(buffer_size,), dtype=np.uint32)

# Padding Inputbuffer
input_buffer[:] = pad_frame

# Laden der Daten in Inputbuffer
input_buffer[: data_size] = input_data
```

Die Datenübertragung wird mithilfe der Methode *transfer()* sowohl für den Sende- als auch für den Empfangskanal gestartet. Dadurch wird der Übertragungsvorgang an den DMA angestoßen. Nach dem Start der Übertragung wird mit *wait()* auf den Abschluss des Transfers gewartet. Während dieser Phase ist die DMA-Schnittstelle blockiert,

```
# Senden und Empfangen der Daten
dma.sendchannel.transfer(input_buffer)
dma.recvchannel.transfer(output_buffer)
dma.sendchannel.wait()
dma.recvchannel.wait()
```

Nach dem Empfang der Daten über den DMA werden diese wieder in das ursprüngliche Format zurückkonvertiert. Falls der Eingabepuffer aufgrund von Padding mit Nullen aufgefüllt wurde, werden diese nach dem Empfang anhand der bekannten Originallänge entfernt, sodass nur die tatsächlich relevanten Audiodaten weiterverarbeitet werden.

```
# Umrechnen der Empfangenen Daten
output_data = np.array(output_buffer[: data_size]).view(np.int32)
```

Öffnen und Einlesen von Wav-Dateien

Zur Verarbeitung von Audiodaten werden zunächst Wav-Dateien geöffnet und eingelesen. Dies geschieht über die Funktion `read_wav()`, die mithilfe des Python-Moduls `wave` Zugriff auf die Audiodaten ermöglicht. Die Funktion basiert auf einem Beispiel aus den PYNQ-Audio-Demonstrationen, die auf dem Board vorinstalliert sind, und wurde entsprechend angepasst. Beim Einlesen der Datei werden zunächst grundlegende Metainformationen wie Anzahl der Kanäle, Abtastrate, Anzahl der Frames sowie die Sample-Breite ausgelesen. Anschließend werden die Rohdaten byteweise in ein Array geladen und abhängig von der Kanalanzahl und Samplegröße korrekt umstrukturiert. Am Ende gibt die Funktion sowohl die Audiodaten als auch die zugehörigen Metadaten (Frame-Anzahl, Kanalzahl, Samplingrate, Sample-Breite) zurück.

```
def read_wav(wav_path):
    with wave.open(wav_path, 'r') as wav_file:
        raw_frames = wav_file.readframes(-1)
        num_frames = wav_file.getnframes()
        num_channels = wav_file.getnchannels()
        sample_rate = wav_file.getframerate()
        sample_width = wav_file.getsampwidth()

    temp_buffer = np.empty((num_frames, num_channels, 4), dtype=np.uint8)
    raw_bytes = np.frombuffer(raw_frames, dtype=np.uint8)
    temp_buffer[:, :, :sample_width] = raw_bytes.reshape(-1, num_channels,
                                                         sample_width)

    temp_buffer[:, :, sample_width:] = \
    (temp_buffer[:, :, sample_width-1:sample_width] >> 7) * 255
    frames = temp_buffer.view('<i4').reshape(temp_buffer.shape[:-1])

    print("Frames:", len(frames), "Channels:", num_channels, "Sample Rate:", sample_rate, "Sample W")
    return frames, num_frames, num_channels, sample_rate, sample_width
```

Schreiben und Speichern von Wav-Dateien

Nach der Verarbeitung der Audiodaten können die Ergebnisse als Datei gespeichert werden. Dies geschieht über die Funktion `save_to_24bit_wav()`, die die beiden Audiokanäle entgegennimmt und diese gemeinsam in eine Stereo-WAV-Datei schreibt. Die Funktion ist in ihrer Struktur an `read_wav()` angelehnt und übernimmt das Schreiben der Daten im passenden Format. Das Schreiben erfolgt über das Python-Modul `wave`, wobei die Daten als flache Bytefolge übergeben werden. Die resultierende Datei kann anschließend direkt im Notebook abgespielt oder zur externen Weiterverarbeitung verwendet werden.

```
def save_to_24bit_wav(chan_l, chan_r, sample_rate, path):
    frames = np.stack((chan_l, chan_r), axis=1)
    max_val = 2**23 # 24-bit max signed int
    frames = np.clip(frames, -1.0, 1.0)
    frames_int = (frames * max_val).astype(np.int32)

    # In Bytes umwandeln
    temp_bytes = frames_int.reshape((*frames.shape, 1)).view(np.uint8)
    raw_bytes = temp_bytes[:, :, :3].reshape(-1)

    with wave.open(path, 'wb') as wav_out:
        wav_out.setnchannels(frames.shape[1])
        wav_out.setsampwidth(3) # 24-bit
        wav_out.setframerate(sample_rate)
        wav_out.writeframes(raw_bytes.tobytes())
```

Paketaufteilung

Wie bereits bekannt, ist die Puffergröße der Filter-IP-Cores auf 2^{20} Bit festgelegt. Nach dem Einlesen einer .wav-Datei kann es jedoch vorkommen, dass die Audiodaten (Frames) länger sind als der verfügbare Puffer. Um dennoch eine korrekte Filterung beliebiger Signallängen zu ermöglichen, werden die Daten vor der Übertragung in kleinere Teilstücke zerlegt. Dies erfolgt über die Funktion `Split2Packets()`, die das Array mit den Audiodaten in mehrere Pakete unterteilt, jeweils mit einer maximalen Größe, die dem festgelegten Puffer entspricht.

```
def Split2Packets(data, packet_size):
    packets = []
    for i in range(0, len(data), packet_size):
        packet = data[i:i+packet_size]
        packets.append(packet)
    return packets
```

Die Funktion *send2receive()* nutzt zunächst *Split2Packets()*, um die Eingangsdaten in kleinere Teilstücke aufzuteilen. Für jedes dieser Datenpakete wird anschließend die Übertragung mithilfe der Funktion *Transmission()* durchgeführt. Die dabei empfangenen, gefilterten Daten werden jeweils aneinandergelagert und in einem Outputarray gesammelt. Auf diese Weise entsteht am Ende ein vollständiges gefiltertes Signal, das dieselbe Länge wie das ursprüngliche Eingangsframe besitzt.

```
def send2receive(Data_In, dma, dma_send, dma_recv):
    ip_buffer = 2**20
    # Filtern Kanal
    Data_Out = []

    # Zerteilung und Übertragung in Paketen
    Packets = Split2Packets(Data_In, ip_buffer)
    anz_trans = 0
    for packet in Packets:
        result = Transmission(packet, ip_buffer, dma, dma_send, dma_recv)
        Data_Out.extend(result)
        anz_trans = anz_trans + 1
    print(anz_trans, "Transmissions")
    return Data_Out
```

vollständige Filter-Anwendung

Die Funktion *UseFilter()* bildet den zentralen Ablauf zur Anwendung eines digitalen Filters auf eine Audiodatei. Sie automatisiert dabei alle notwendigen Schritte, vom Einlesen der Datei bis zur Speicherung des gefilterten Ergebnisses.

Zu Beginn wird die gewählte Filterinstanz ausgewählt. Anschließend wird die angegebene .wav-Datei über die Funktion *read_wav()* geöffnet. Dabei werden sowohl die Audiodaten der beiden Kanäle als auch relevante Metainformationen ausgelesen. Anschließend erfolgt eine Normierung der Audiodaten um eine saubere Signalverarbeitung sicherzustellen.

Falls die Samplerate der Eingangsdaten nicht 48 kHz beträgt, was häufig bei Standard-Audiodateien mit 44,1 kHz der Fall ist, wird automatisch ein Upsampling auf 48 kHz durchgeführt. Dies ist erforderlich, da die Filter für diese Abtastrate ausgelegt wurden. Auf diese Weise wird das richtige Verhalten des Filters sichergestellt.

Im nächsten Schritt wird das normierte Audiosignal für jeden Kanal getrennt an den zuvor ausgewählten Filter übergeben. Die Übertragung und Filterung wird über die Funktion *send2receive()* abgewickelt, die das Signal in Pakete unterteilt, überträgt, empfängt und anschließend wieder zusammensetzt.

Zuletzt werden die gefilterten Daten beider Kanäle mithilfe der Funktion *save_to_24bit_wav()* wieder zu einer Stereo-WAV-Datei zusammengeführt und unter dem angegebenen Ausgabedateinamen gespeichert.

```

def UseFilter(in_Name, out_Name, Filter):
    import time
    start = time.time()
    dma = Filter[0]
    dma_send = Filter[1]
    dma_recv = Filter[2]
    [frames, num_frames, channels, Fs, Fw] = read_wav(in_Name)
    # Read Data
    data_l = Normierung(frames[:,0])
    data_r = Normierung(frames[:,1])
    if Fs != 48000:
        data_l = resample_poly(data_l, 48000, Fs)
        data_r = resample_poly(data_r, 48000, Fs)
        print("up-sampled: 44.1 -> 48kHz")
        Fs = 48000
    print("Start Sending left Channel")
    Data_Out_L = send2receive(data_l, dma, dma_send, dma_recv)
    print("Start Sending right Channel")
    Data_Out_R = send2receive(data_r, dma, dma_send, dma_recv)
    print("Saving File")
    save_to_24bit_wav(Data_Out_L, Data_Out_R, Fs, out_Name)
    end = time.time()
    print("Finished")
    print(f"Dauer: {end - start:.2f} Sekunden")

```

Kaskadierung der Filter

Neben der Standardfunktion *UseFilter()* steht eine erweiterte Variante zur Verfügung, die es ermöglicht, dasselbe Audiosignal mehrfach durch denselben Filter zu leiten. *UseFilterCascade()* erlaubt es, über einen Parameter festzulegen, wie oft die Filterung wiederholt werden soll. Hintergrund ist, dass in jedem Filter-IP-Core lediglich ein einzelnes Biquad-Segment implementiert ist. Durch die wiederholte Anwendung desselben Filters lässt sich somit eine Kaskadierung mehrerer Biquads realisieren. Dies führt zu einer Erhöhung der effektiven Filterordnung und damit zu einer steileren Flankensteilheit im Frequenzgang, ein Effekt, der deutlich hör- und sichtbar wird.

Diese Funktion dient ausschließlich zu Demonstrationszwecken, bei der gezeigt werden soll, wie sich die Eigenschaften eines Filters durch mehrfache Anwendung bzw. Kaskadierung verändern lassen.

Fazit und Ausblick

Die Umsetzung digitaler Filter auf FPGA-Hardware ist komplex und erfordert in der Regel fundierte Kenntnisse in VHDL sowie ein tiefes Verständnis der Zielarchitektur. Der modellbasierte Workflow mit MATLAB/Simulink und dem *HDL Coder* reduziert diesen Aufwand, da aus einem Simulink-Modell automatisch synthesesfähiger VHDL-Code generiert wird. Dadurch können typische Fehlerquellen frühzeitig erkannt und behoben werden. Besonders vorteilhaft ist die strukturierte Anleitung durch den *HDL Workflow Advisor*, der den Code schrittweise an die Zielplattform anpasst. Anforderungen wie AXI4-Stream-Kompatibilität oder Taktmanagement werden automatisch berücksichtigt. Auf dieser Grundlage konnten alle vier IIR-Filter als parametrisierte Simulink-Modelle entworfen und als IP-Cores generiert werden.

Die Integration in Vivado erfolgte über den *Block Design Editor*, dessen grafische Oberfläche eine intuitive Verbindung der Module ermöglicht. Besonders in der iterativen Entwicklungsphase erwies sich dieser modulare Aufbau als praktikabel. Ein geplanter Echtzeitbetrieb über die I²S-Schnittstelle des ADAU1761 scheiterte jedoch an Kompatibilitätsproblemen zwischen den automatisch generierten Filter-IP-Cores und den bestehenden I²S-Komponenten. Die genaue Ursache konnte aufgrund der abstrahierten Struktur der HDL-Coder-Ausgabe nicht eindeutig identifiziert werden.

Stattdessen wurde eine alternative Lösung zur Verarbeitung zuvor aufgezeichneter Audiodaten realisiert. Dabei wurde die Fähigkeit des PYNQ-Z2 genutzt, analoge Quellen über den integrierten Audio-Codec aufzunehmen. Das Design in Vivado konnte entsprechend angepasst werden, um diese Funktionalität zu unterstützen. Die Filterung erfolgt anschließend über ein eigens entwickeltes Jupyter-Notebook, das auf dem PYNQ-Z2 ausgeführt wird. Die Benutzeroberfläche ermöglicht eine unkomplizierte Anwendung der digitalen Filter auf gespeicherte Audiosignale, sowohl aus digitalen als auch aus analogen Quellen.

Trotz einzelner technischer Herausforderungen konnte somit ein funktionales Gesamtsystem realisiert werden, das sich hervorragend für Demonstrations- und Lehrzwecke eignet.

Für eine zukünftige Weiterentwicklung des Projekts bietet sich die Möglichkeit, an den etablierten Filterdesigns anzuknüpfen und diese funktional zu erweitern. Ein vielversprechender Ansatz wäre die Ergänzung eines Filters um eine AXI4-Lite-Schnittstelle, über die sich Register zur dynamischen Übergabe von Filterkoeffizienten ansteuern lassen. In Kombination mit einer funktionierenden Echtzeitfilterung könnte so ein anpassbarer Echtzeitfilter realisiert werden, dessen Charakteristik sich während des Betriebs flexibel verändern lässt. Dies würde das System nicht nur deutlich vielseitiger machen, sondern auch neue Einsatzmöglichkeiten in interaktiven oder adaptiven Audioszenarien erschließen.